



10-423/10-623/10-723 Generative AI

Machine Learning Department
School of Computer Science
Carnegie Mellon University

Transformer Language Models

Aran Nayebi & Matt Gormley
Lecture 2

Reminders

- **Homework 0: PyTorch + Weights & Biases**
 - Out: Wed, Jan 14
 - Due: Mon, Jan 26 at 11:59pm
 - Two parts:
 1. written part to Gradescope
 2. programming part to Gradescope
 - Unique policy for this assignment: For Slot A submissions (human only – no AI use!), we will grant (essentially) any reasonable and late add extension requests, but you must request one. **However, your HWO has to be in *before* we finish grading!**
 - Any extension you ask for on HWO will run into the release window of HW1 which does not offer this much flexibility.

Some History of...

LARGE LANGUAGE MODELS

Noisy Channel Models

- Prior to 2017, two tasks relied heavily on language models:
 - speech recognition
 - machine translation
- Definition: a **noisy channel model** combines a *transduction model* (probability of converting $\mathbf{y}$ to $\mathbf{x}$) with a *language model* (probability of $\mathbf{y}$)

$$\hat{\mathbf{y}} = \operatorname{argmax}_{\mathbf{y}} p(\mathbf{y} \mid \mathbf{x}) = \operatorname{argmax}_{\mathbf{y}} \underbrace{p(\mathbf{x} \mid \mathbf{y})}_{\text{transduction model}} \underbrace{p(\mathbf{y})}_{\text{language model}}$$

- **Goal:** to recover $\mathbf{y}$ from $\mathbf{x}$
 - For speech: $\mathbf{x}$ is acoustic signal, $\mathbf{y}$ is transcription
 - For machine translation: $\mathbf{x}$ is sentence in source language, $\mathbf{y}$ is sentence in target language

Large (n-Gram) Language Models

- The earliest (truly) large language models were n-gram models
- Google n-Grams:
 - 2006: first release, English n-grams
 - trained on **1 trillion tokens** of web text (95 billion sentences)
 - included 1-grams, 2-grams, 3-grams, 4-grams, and 5-grams
 - 2009 – 2010: n-grams in Japanese, Chinese, Swedish, Spanish, Romanian, Portuguese, Polish, Dutch, Italian, French, German, Czech

English n-gram model is ~**3 billion parameters**

Number of unigrams:	13,588,391
Number of bigrams:	314,843,401
Number of trigrams:	977,069,902
Number of fourgrams:	1,313,818,354
Number of fivegrams:	1,176,470,663

serve as the incoming 92
serve as the incubator 99
serve as the independent 794
serve as the index 223
serve as the indication 72
serve as the indicator 120
serve as the indicators 45
serve as the indispensable 111
serve as the indispensible 40
serve as the individual 234
serve as the industrial 52
serve as the industry 607

accessoire Accessoires </S> 515
accessoire Accord i-CTDi 65
accessoire Accra accu 312
accessoire Acheter cet 1402
accessoire Ajouter au 160
accessoire Amour Beauté 112
accessoire Annuaire LOEIL 49
accessoire Architecture artiste 531
accessoire Attention : 44

惯例 为 电影 创作 52
惯例 为 的 是 95
惯例 为 目标 职位 49
惯例 为 确保 合作 69
惯例 为 确保 重组 213
惯例 为 科研 和 55
惯例 为 统称 </s> 183
惯例 为 维 和 50
惯例 为 自己 的 43
惯例 为 艺术类 学院 44
惯例 为 避免 侵权 148

Large (n-Gram) Language Models

- The earliest (truly) large language models were n-gram models
- Google n-Grams:
 - 2006: first release, English n-grams
 - trained on **1 trillion tokens** of web text (95 billion sentences)
 - included 1-grams, 2-grams, 3-grams, 4-grams, and 5-grams
 - 2009 – 2010: n-grams in Japanese, Chinese, Swedish, Spanish, Romanian, Portuguese, Polish, Dutch, Italian, French, German, Czech

English n-gram model is ~**3 billion parameters**



Number of unigrams:	13,588,391
Number of bigrams:	314,843,401
Number of trigrams:	977,069,902
Number of fourgrams:	1,313,818,354
Number of fivegrams:	1,176,470,663

Q: Is this a large training set?

A: Yes!

Q: Is this a large model?

A: Yes!

serve as the individual 234
serve as the industrial 52
serve as the industry 607

oire Accessoires
oire Accord i-
oire Accra acc
oire Acheter d
oire Ajouter d
oire Amour Bea
oire Annuaire
oire Architect
oire Attention

惯例 为 艺术类 学院 44
惯例 为 避免 侵权 148
惯例 为 避免 侵权

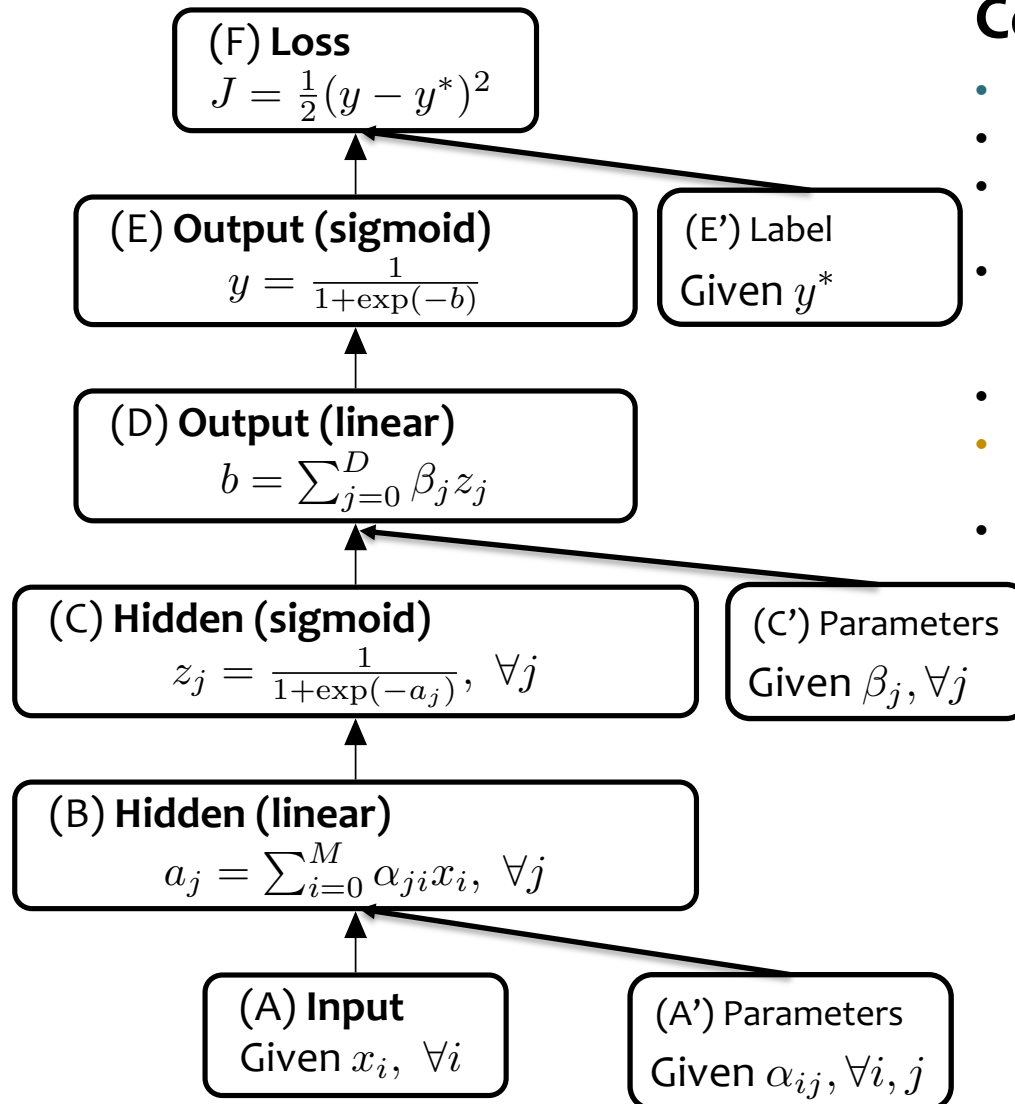
How large are LLMs?

Comparison of some recent **large language models** (LLMs)

Model	Creators	Year of release	Training Data (# tokens)	Model Size (# parameters)
GPT-2	OpenAI	2019	~10 billion (40Gb)	1.5 billion
GPT-3	OpenAI	2020	300 billion	175 billion
PaLM	Google	2022	780 billion	540 billion
Chinchilla	DeepMind	2022	1.4 trillion	70 billion
LaMDA (cf. Bard)	Google	2022	1.56 trillion	137 billion
LLaMA	Meta	2023	1.4 trillion	65 billion
LLaMA-2	Meta	2023	2 trillion	70 billion
GPT-4 & GPT-5	OpenAI	2023 & 2025	?	? (1.76 trillion & 2-5 trillion)
Gemini (Ultra)	Google	2023	?	? (1.5 trillion)
LLaMA-3	Meta	2024	15 trillion	405 billion

FORGETFUL RNNS

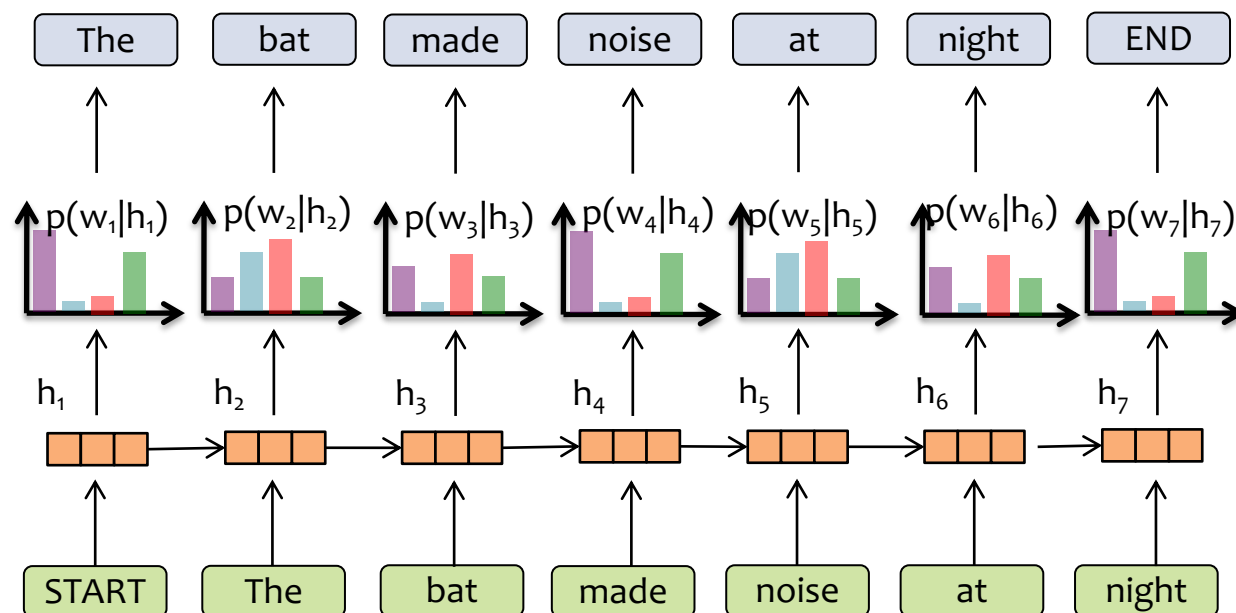
Ways of Drawing Neural Networks



Computation Graph

- The diagram represents an algorithm
- Nodes are **rectangles**
- One node per **intermediate variable in the algorithm**
- Node is labeled with the **function** that it computes (inside the box) and also the **variable** name (outside the box)
- Edges are directed
- Edges do not have labels (since they don't need them)
- For neural networks:
 - Each **intercept term** should appear as a node (if it's not folded in somewhere)
 - Each parameter should appear as a node
 - Each constant, e.g. a true label or a feature vector should appear in the graph
 - It's **perfectly fine** to include the loss

RNN Language Model



Key Idea:

- (1) convert all previous words to a **fixed length vector**
- (2) define distribution $p(w_t \mid f_{\theta}(w_{t-1}, \dots, w_1))$ that conditions on the vector $\mathbf{h}_t = f_{\theta}(w_{t-1}, \dots, w_1)$

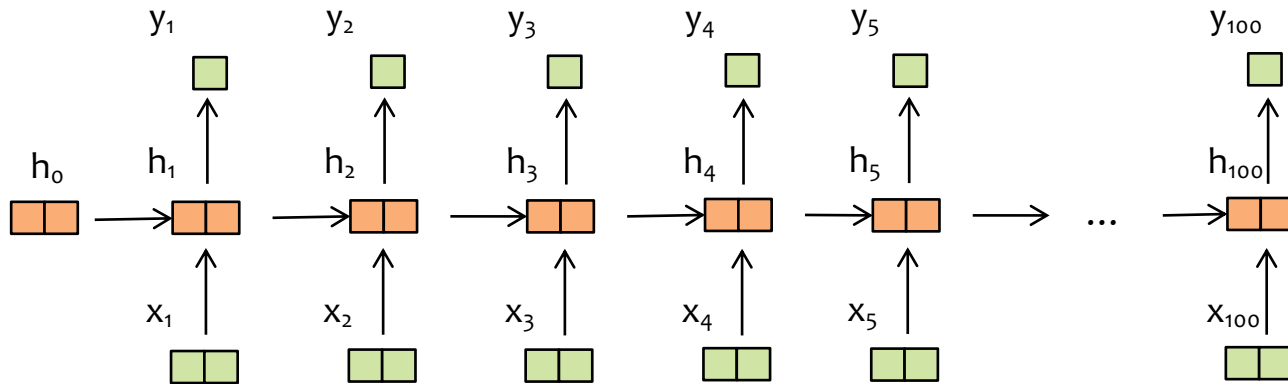
RNNs and Forgetting

Suppose we want an RNN over binary vectors of length 2 that can remember whether or not it has seen a value of 1 in both input positions.

$$\mathbf{h}_t = \sigma(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h)$$

$$y_t = \text{sign}(\mathbf{W}_{yh}\mathbf{h}_t + b_y)$$

$$\sigma \equiv \text{identity}$$



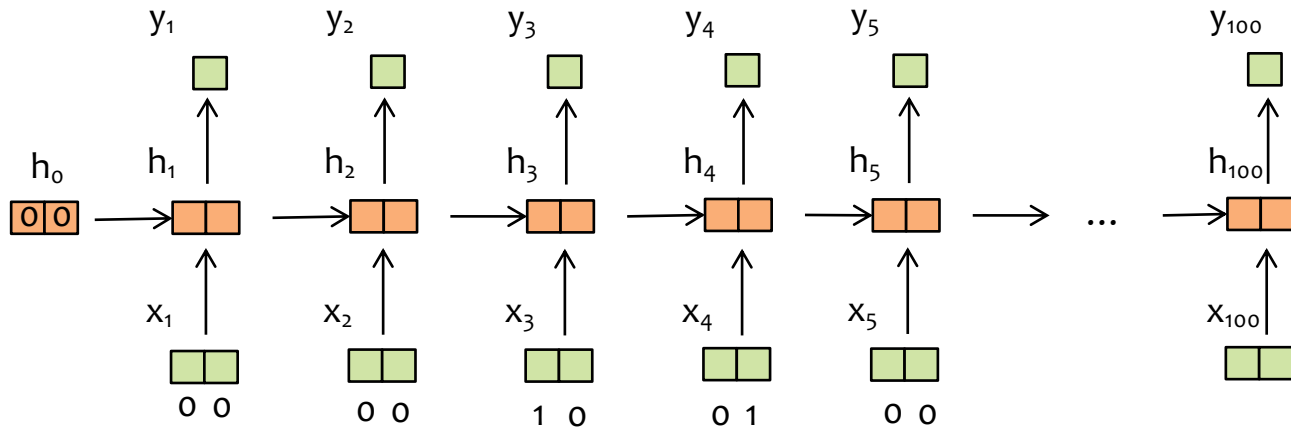
RNNs and Forgetting

Suppose we want an RNN over binary vectors of length 2 that can remember whether or not it has seen a value of 1 in both input positions.

$$\mathbf{h}_t = \sigma(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h)$$

$$y_t = \text{sign}(\mathbf{W}_{yh}\mathbf{h}_t + b_y)$$

$$\sigma \equiv \text{identity}$$



RNNs and Forgetting

Suppose we want an RNN over binary vectors of length 2 that can remember whether or not it has seen a value of 1 in both input positions.

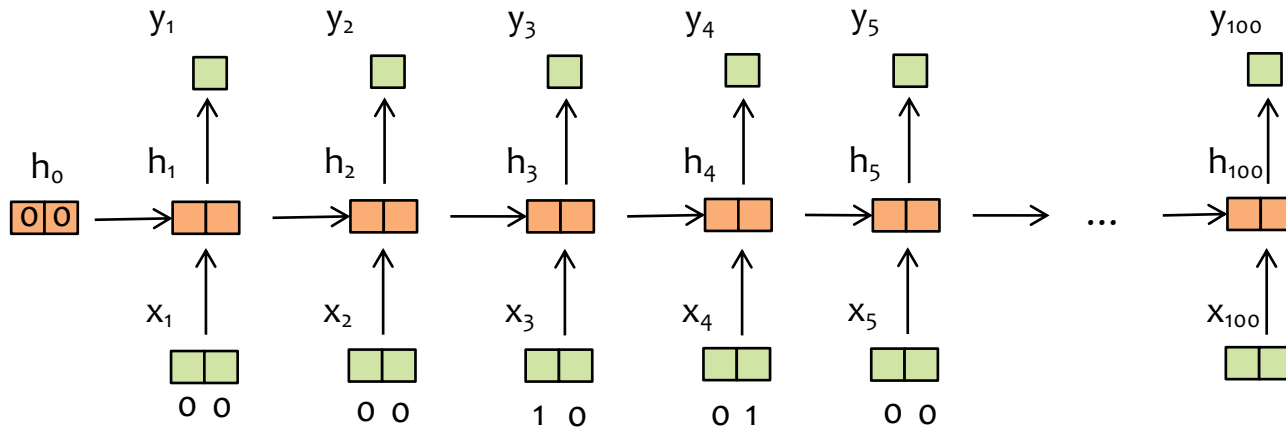
$$\mathbf{h}_t = \sigma(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h)$$

$$y_t = \text{sign}(\mathbf{W}_{yh}\mathbf{h}_t + b_y)$$

$\sigma \equiv \text{identity}$

$$\mathbf{W}_{hx} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \mathbf{W}_{hh} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{bmatrix} \quad \mathbf{b}_h = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\mathbf{W}_{yh} = \begin{bmatrix} \quad \end{bmatrix} \quad \mathbf{b}_y = \begin{bmatrix} \quad \\ \quad \end{bmatrix}$$



RNNs and Forgetting

Suppose we want an RNN over binary vectors of length 2 that can remember whether or not it has seen a value of 1 in both input positions.

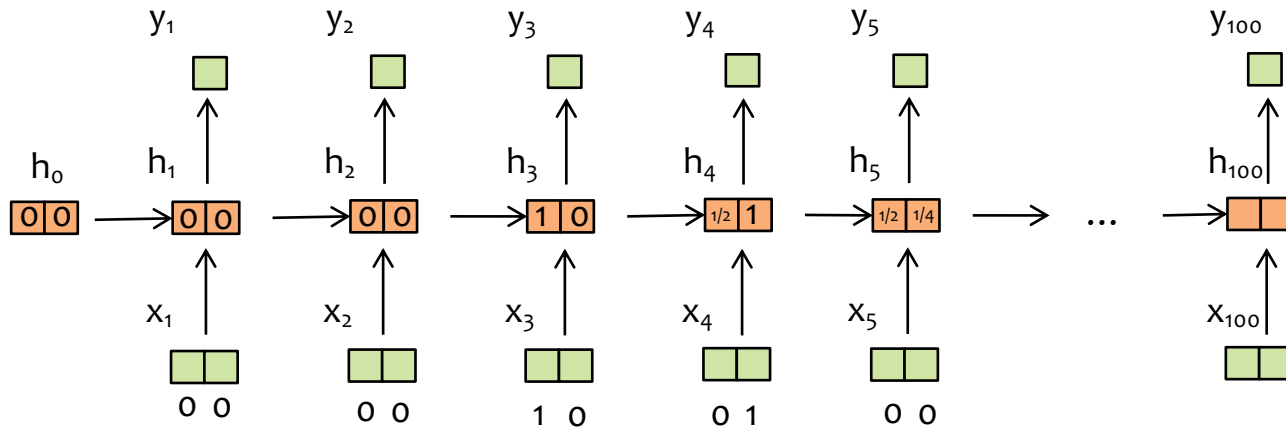
$$\mathbf{h}_t = \sigma(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h)$$

$$y_t = \text{sign}(\mathbf{W}_{yh}\mathbf{h}_t + b_y)$$

$\sigma \equiv \text{identity}$

$$\mathbf{W}_{hx} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \mathbf{W}_{hh} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{bmatrix} \quad \mathbf{b}_h = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\mathbf{W}_{yh} = \begin{bmatrix} \quad \quad \end{bmatrix} \quad \mathbf{b}_y = \begin{bmatrix} \quad \\ \quad \end{bmatrix}$$



RNNs and Forgetting

Suppose we want an RNN over binary vectors of length 2 that can remember whether or not it has seen a value of 1 in both input positions.

$$\mathbf{h}_t = \sigma(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h)$$

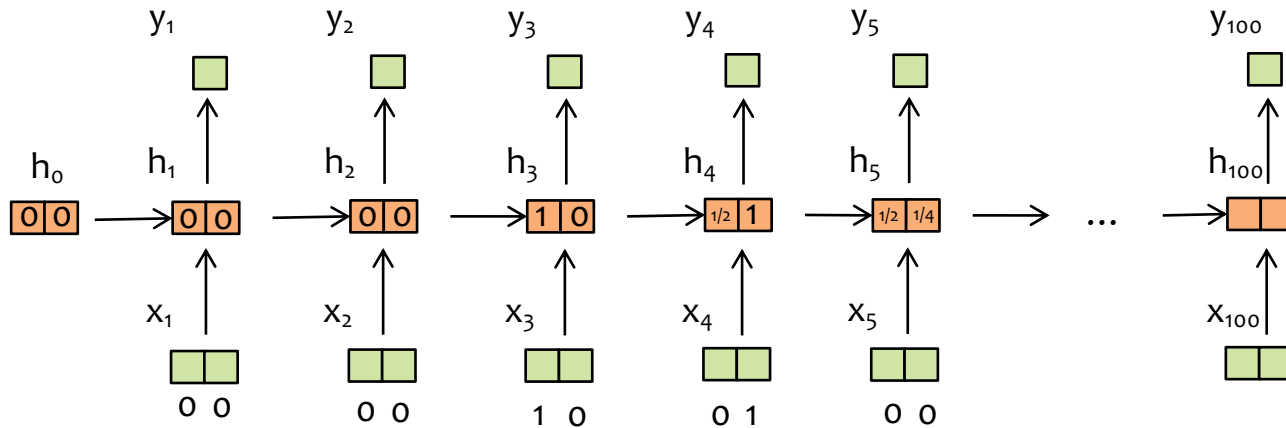
$$y_t = \text{sign}(\mathbf{W}_{yh}\mathbf{h}_t + b_y)$$

$\sigma \equiv \text{identity}$

$$\mathbf{W}_{hx} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \mathbf{W}_{hh} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{bmatrix} \quad \mathbf{b}_h = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\mathbf{W}_{yh} = \begin{bmatrix} ? \\ ? \end{bmatrix} \quad \mathbf{b}_y = \begin{bmatrix} ? \\ ? \end{bmatrix}$$

chosen s.t.



$$y_t = +1 \quad \text{if } h_{t,1} \geq 0.1$$

$$\text{and } h_{t,2} \geq 0.1$$

$$y_t = -1 \quad \text{otherwise}$$

RNNs and Forgetting

Suppose we want an RNN over binary vectors of length 2 that can remember whether or not it has seen a value of 1 in both input positions.

$$\mathbf{h}_t = \sigma(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h)$$

$$y_t = \text{sign}(\mathbf{W}_{yh}\mathbf{h}_t + b_y)$$

$\sigma \equiv \text{identity}$

$$\mathbf{W}_{hx} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \mathbf{W}_{hh} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{bmatrix} \quad \mathbf{b}_h = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

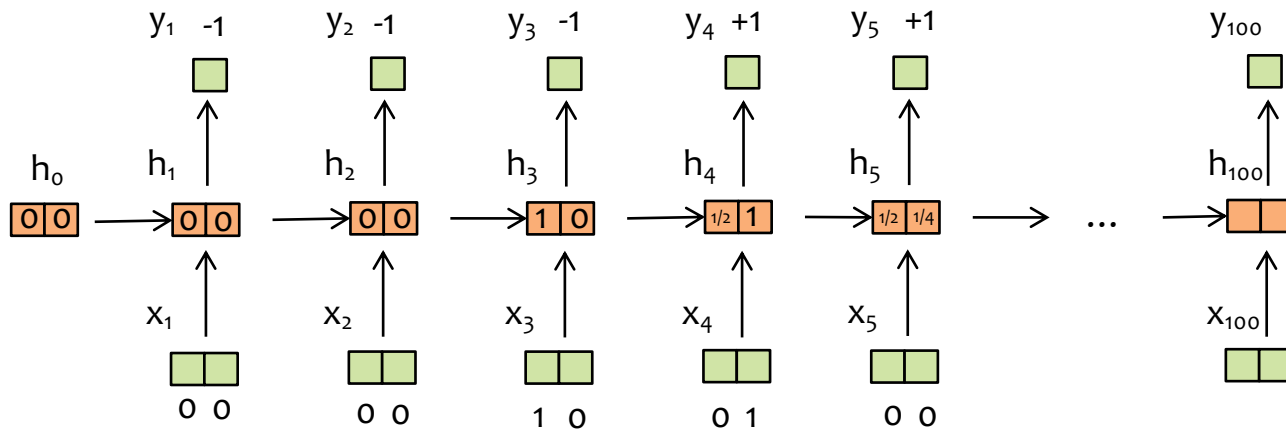
$$\mathbf{W}_{yh} = \begin{bmatrix} ? \\ ? \end{bmatrix} \quad \mathbf{b}_y = \begin{bmatrix} ? \\ ? \end{bmatrix}$$

chosen s.t.

$$y_t = +1 \quad \text{if } h_{t,1} \geq 0.1$$

$$\text{and } h_{t,2} \geq 0.1$$

$$y_t = -1 \quad \text{otherwise}$$



RNNs and Forgetting

Suppose we want an RNN over binary vectors of length 2 that can remember whether or not it has seen a value of 1 in both input positions.

$$\mathbf{h}_t = \sigma(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h)$$

$$y_t = \text{sign}(\mathbf{W}_{yh}\mathbf{h}_t + b_y)$$

$\sigma \equiv \text{identity}$

$$\mathbf{W}_{hx} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \mathbf{W}_{hh} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{bmatrix} \quad \mathbf{b}_h = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

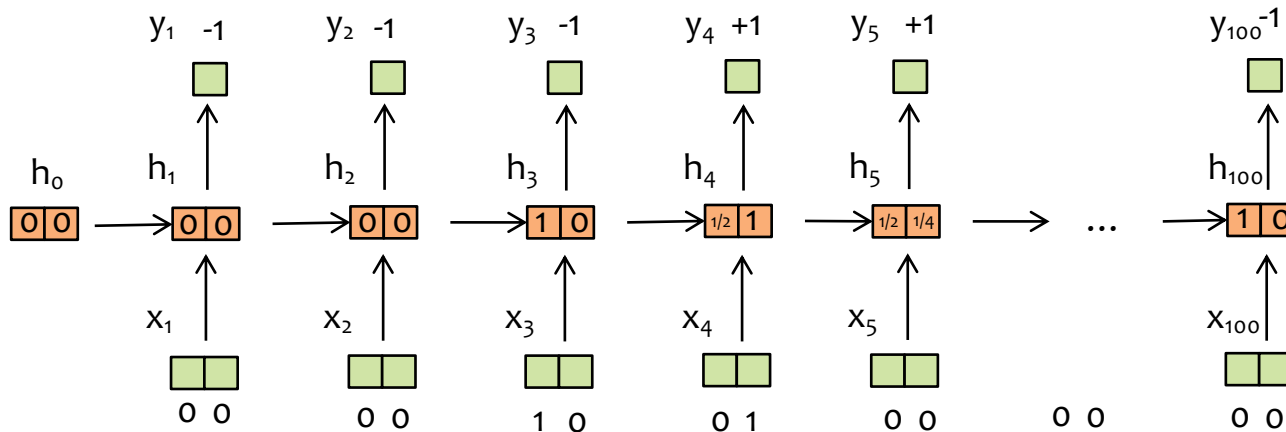
$$\mathbf{W}_{yh} = \begin{bmatrix} ? \\ ? \end{bmatrix} \quad \mathbf{b}_y = \begin{bmatrix} ? \\ ? \end{bmatrix}$$

chosen s.t.

$$y_t = +1 \quad \text{if } h_{t,1} \geq 0.1$$

$$\text{and } h_{t,2} \geq 0.1$$

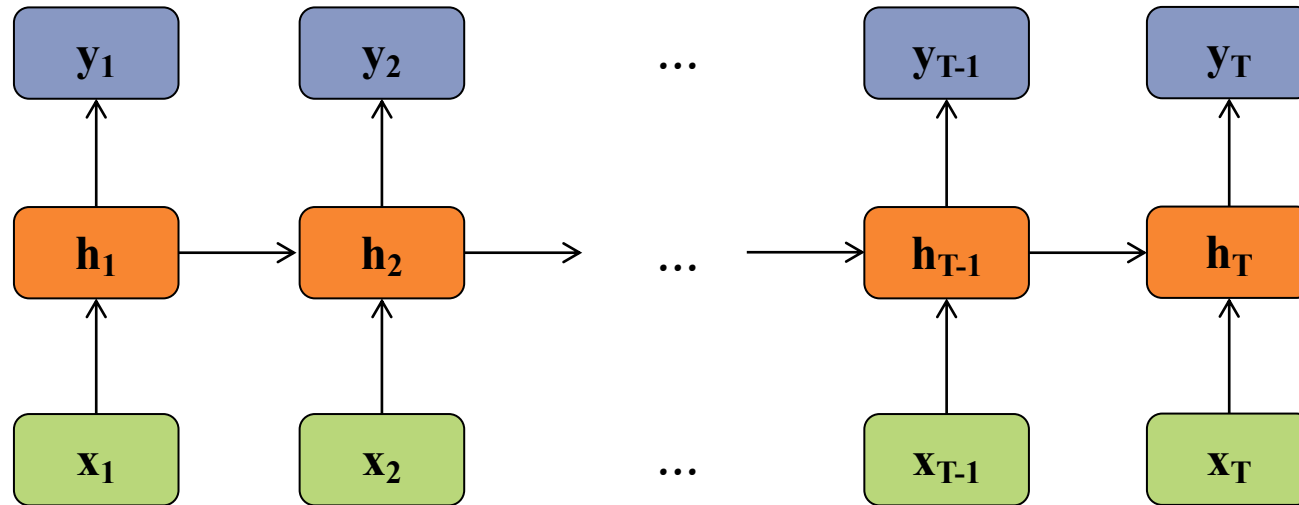
$$y_t = -1 \quad \text{otherwise}$$



Long Short-Term Memory (LSTM)

Motivation:

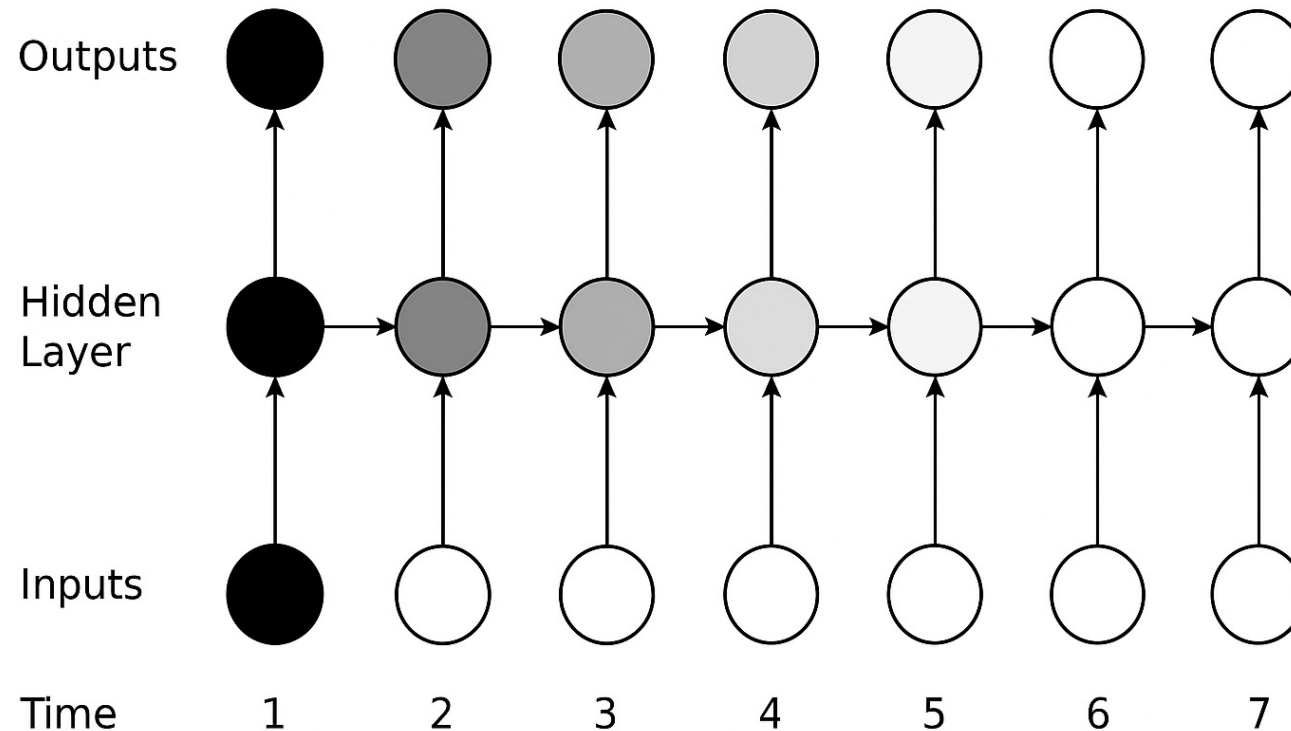
- Standard RNNs have trouble learning long distance dependencies
- LSTMs combat this issue



Long Short-Term Memory (LSTM)

Motivation:

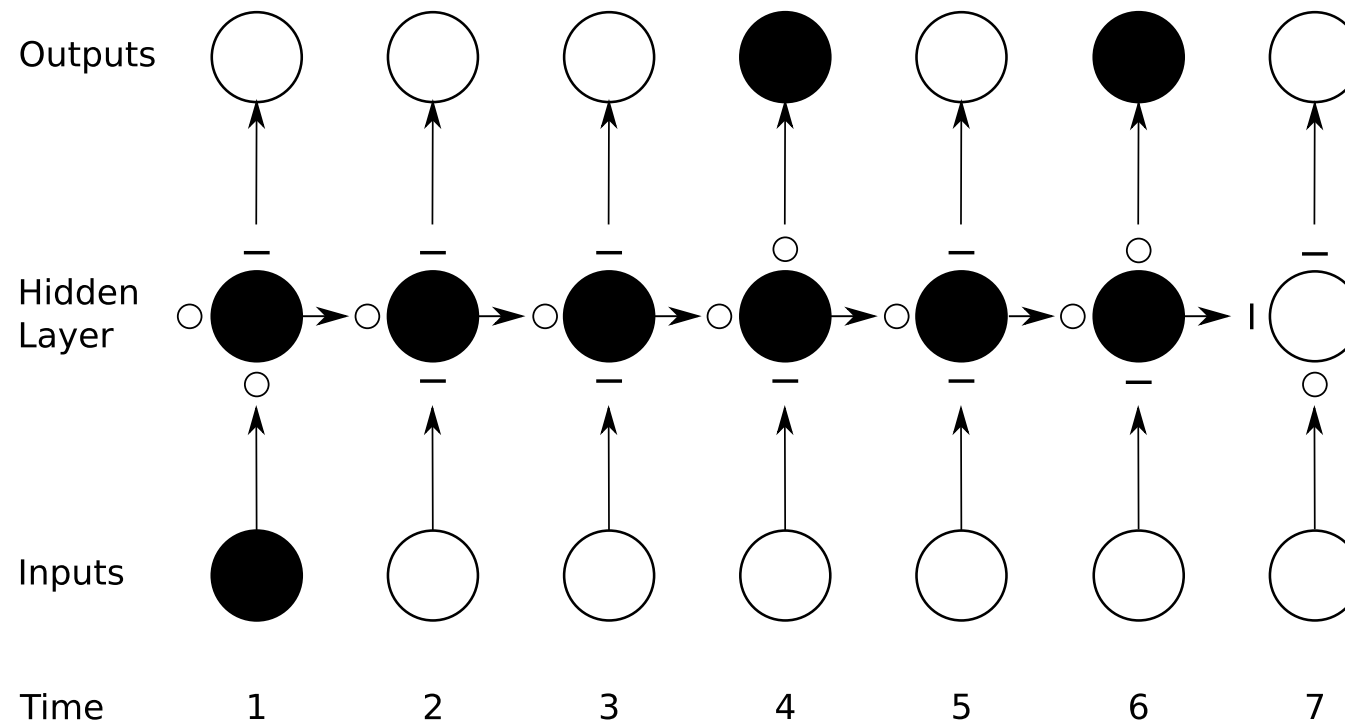
- Vanishing gradient problem for Standard RNNs
- Figure shows sensitivity (darker = more sensitive) to the input at time $t=1$



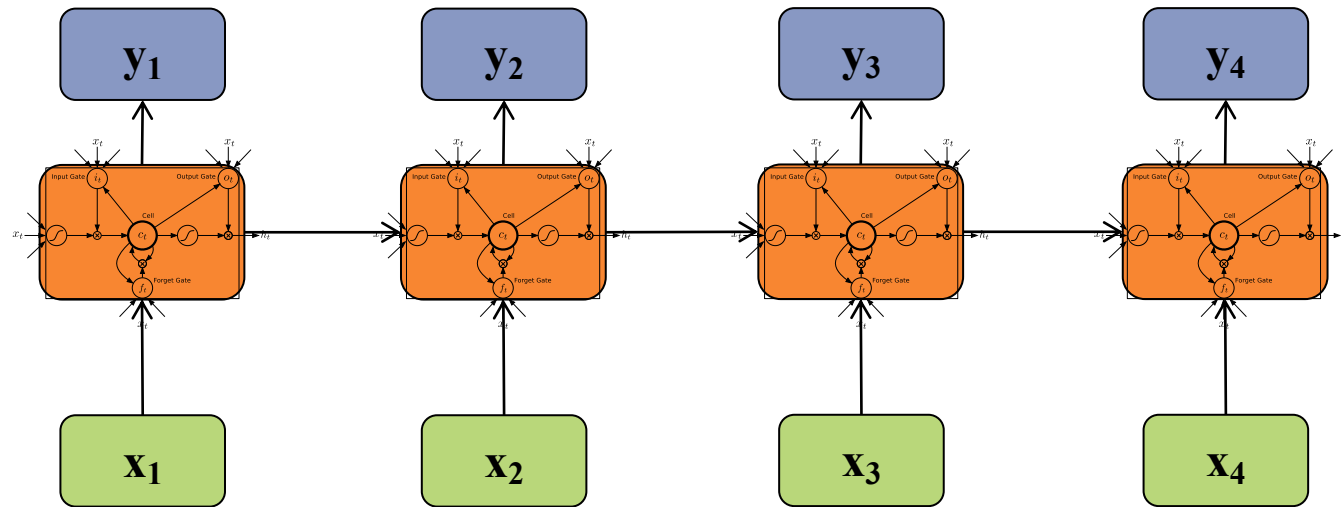
Long Short-Term Memory (LSTM)

Motivation:

- LSTM units have a rich internal structure
- The various “gates” determine the propagation of information and can choose to “remember” or “forget” information

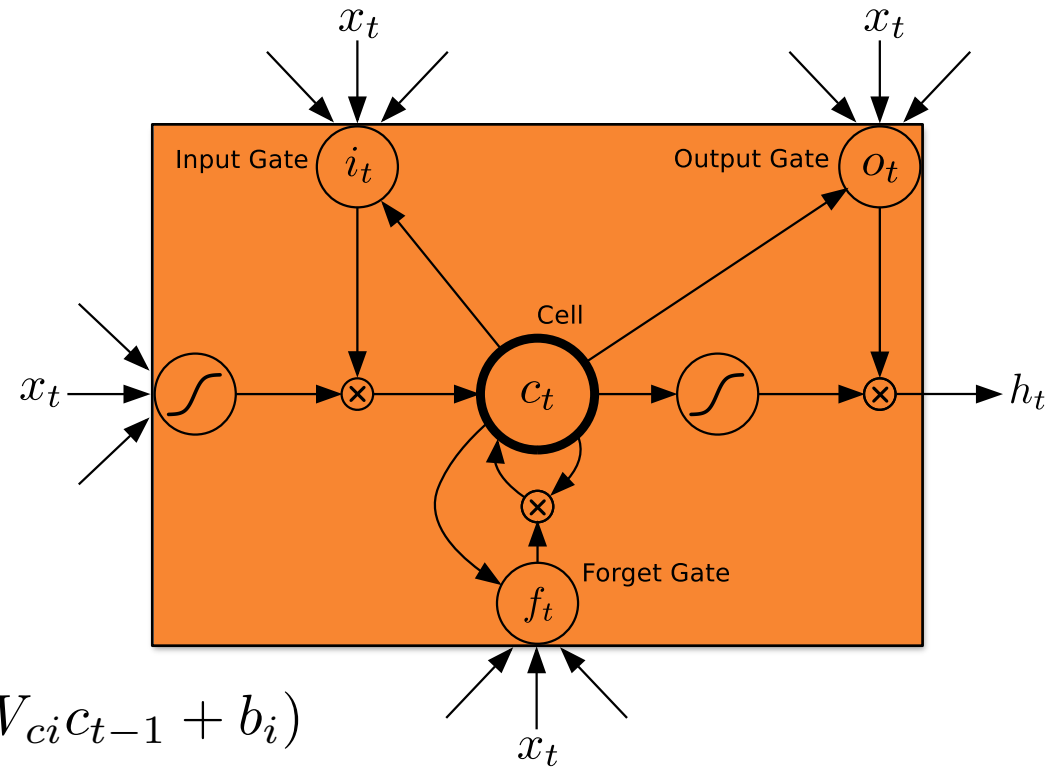


Long Short-Term Memory (LSTM)



Long Short-Term Memory (LSTM)

- **Input gate:** masks out the standard RNN inputs
- **Forget gate:** masks out the previous cell
- **Cell:** stores the input/forget mixture
- **Output gate:** masks out the values of the next hidden state



$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + W_{ci}c_{t-1} + b_i)$$

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + W_{cf}c_{t-1} + b_f)$$

$$c_t = f_t c_{t-1} + i_t \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$$

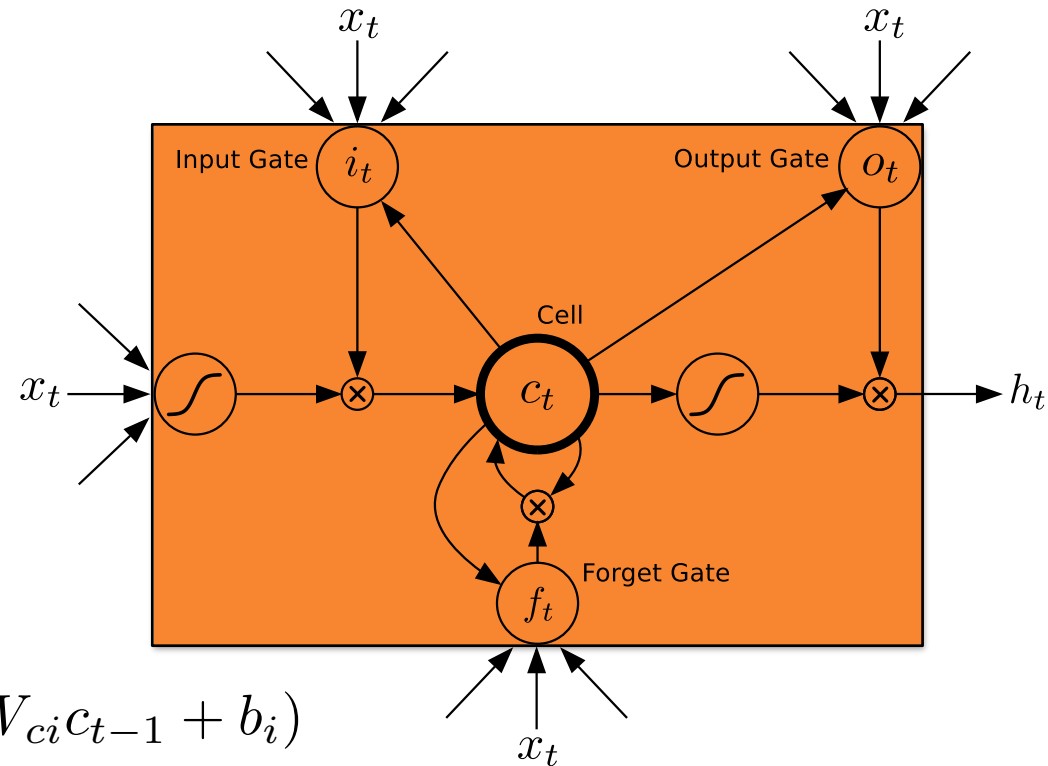
$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + W_{co}c_t + b_o)$$

$$h_t = o_t \tanh(c_t)$$

Figure from (Graves et al., 2013)

Long Short-Term Memory (LSTM)

- **Input gate:** masks out the standard RNN inputs
- **Forget gate:** masks out the previous cell
- **Cell:** stores the input/forget mixture
- **Output gate:** masks out the values of the next hidden state



$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + W_{ci}c_{t-1} + b_i)$$

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + W_{cf}c_{t-1} + b_f)$$

$$c_t = f_t c_{t-1} + i_t \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$$

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + W_{co}c_t + b_o)$$

$$h_t = o_t \tanh(c_t)$$

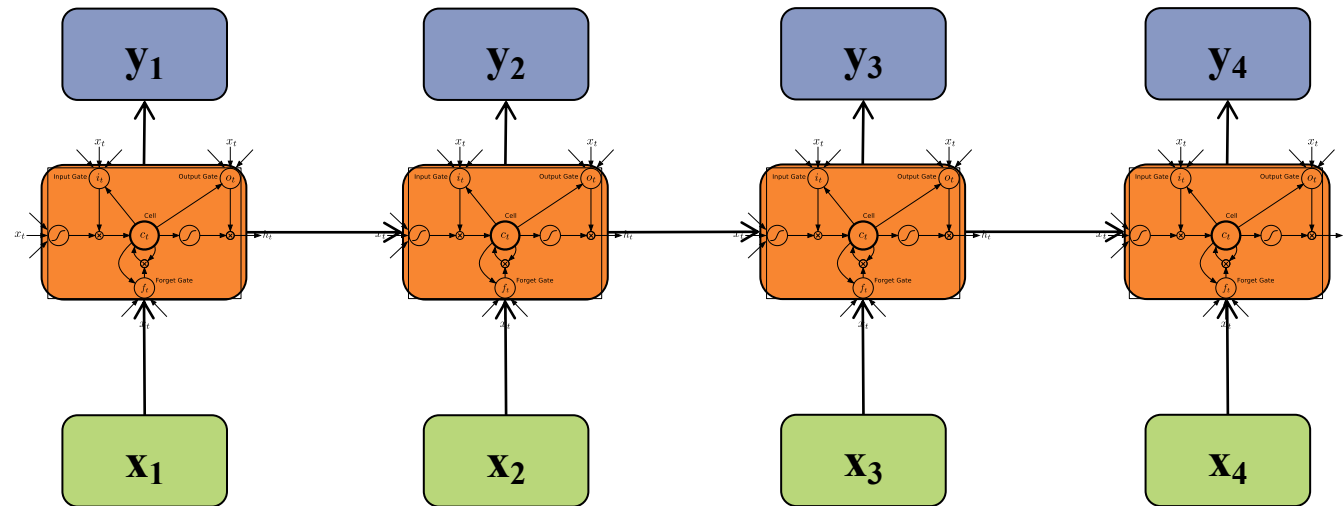
Identical to the Elman's networks hidden state

Figure from (Graves et al., 2013)

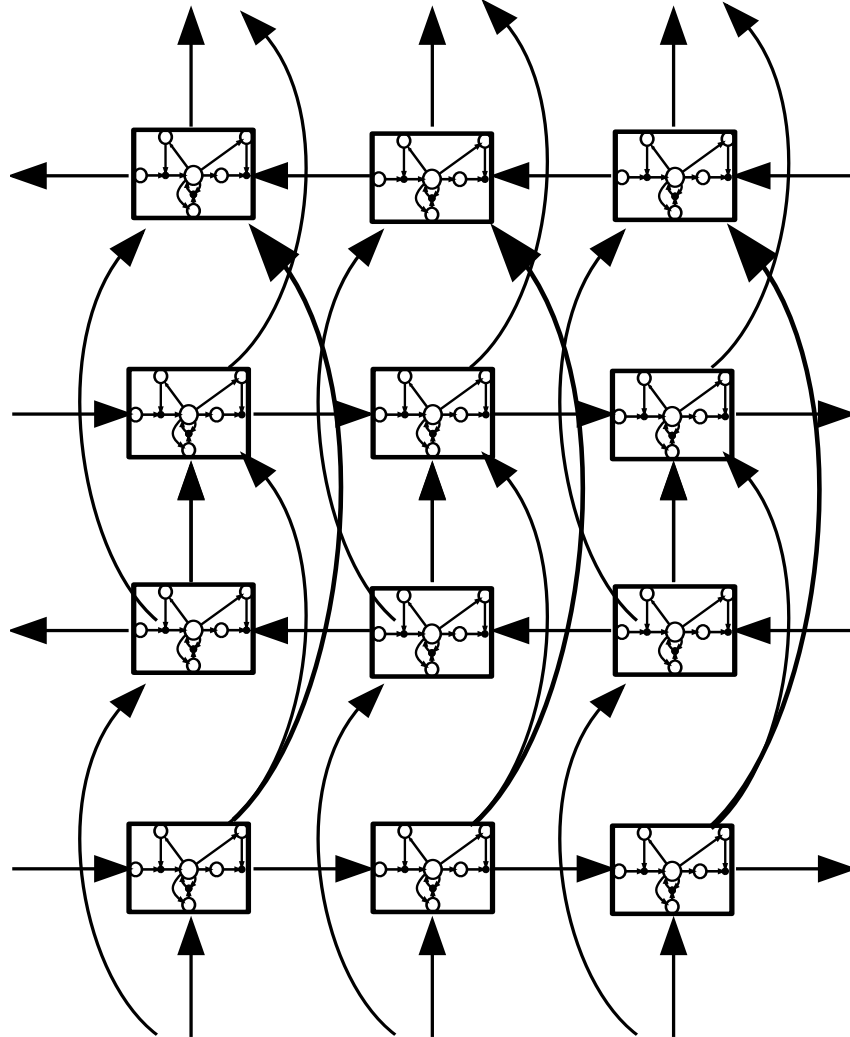
The cell is the LSTM's long term memory, and helps control information flow over time steps

The hidden state is the output of the LSTM cell

Long Short-Term Memory (LSTM)

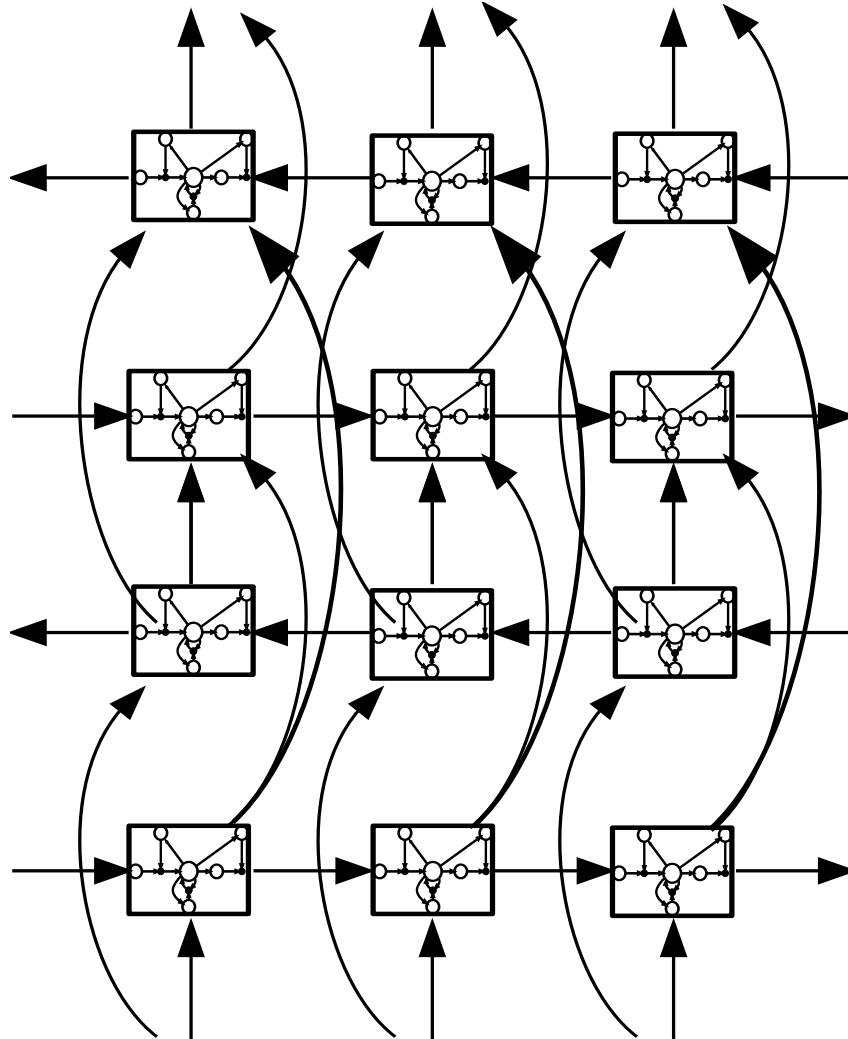


Deep Bidirectional LSTM (DBLSTM)



- Figure: input/output layers not shown
- **Same general topology** as a Deep Bidirectional RNN, but with **LSTM units** in the hidden layers
- No additional **representational power** over DBRNN, but **easier to learn** in practice

Deep Bidirectional LSTM (DBLSTM)



How important is this particular architecture?

Jozefowicz et al. (2015) **evaluated 10,000 different LSTM-like architectures** and found several variants that worked just as well on several tasks.

Why not just use LSTMs for everything?

Everyone did, for a time.

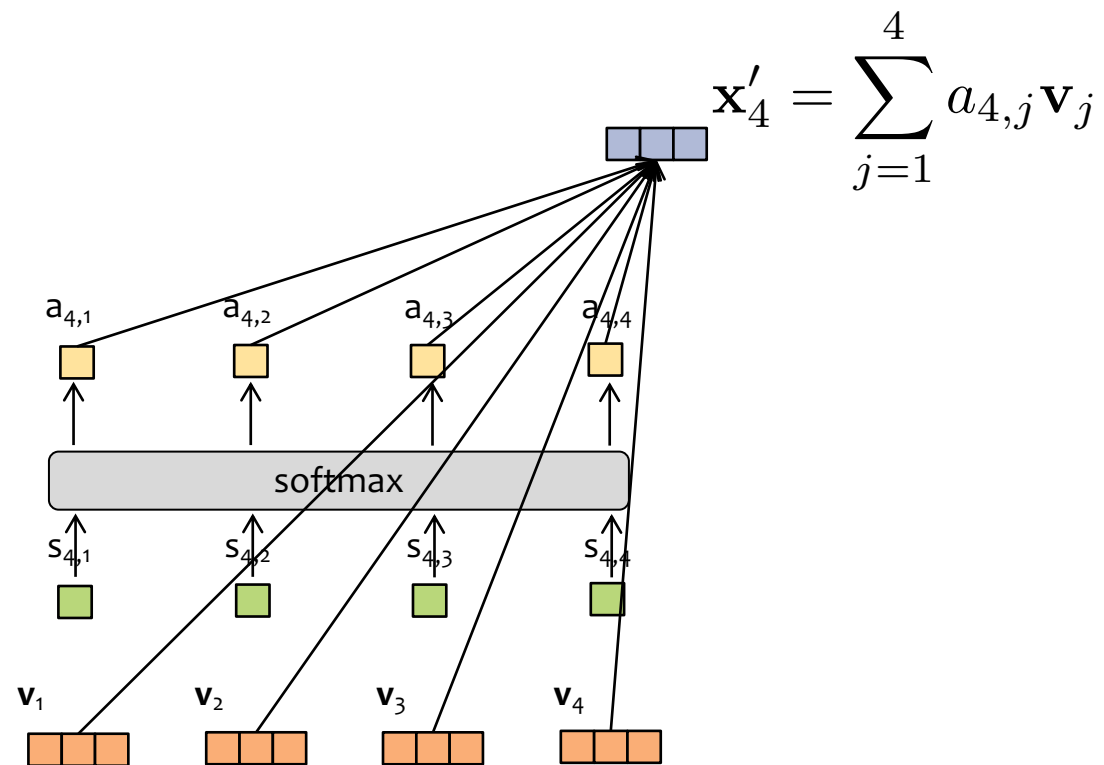
But...

1. They still have **difficulty** with **long-range dependencies**
2. Their computation is **inherently serial**, so can't be easily parallelized on a GPU
3. Even though they (mostly) solve the vanishing gradient problem, they can still suffer from **exploding gradients**

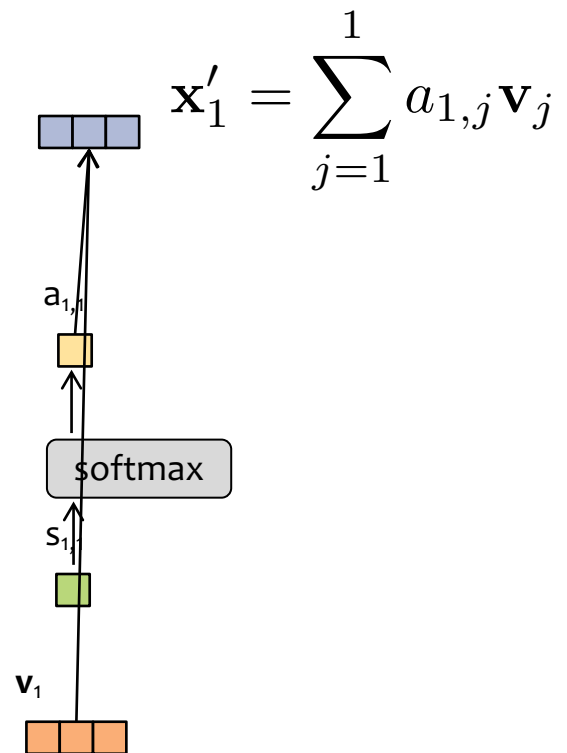
Transformer Language Models

MODEL: GPT

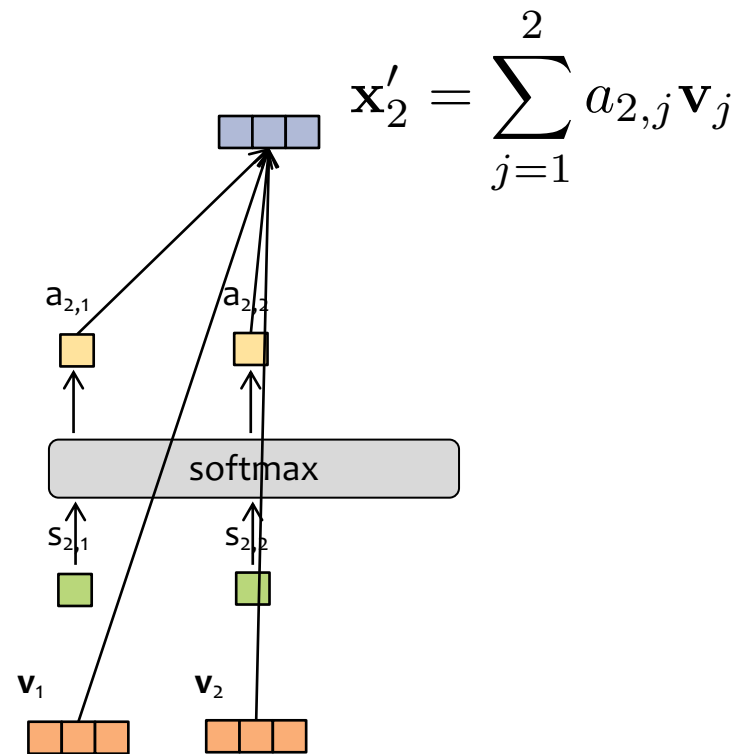
Attention



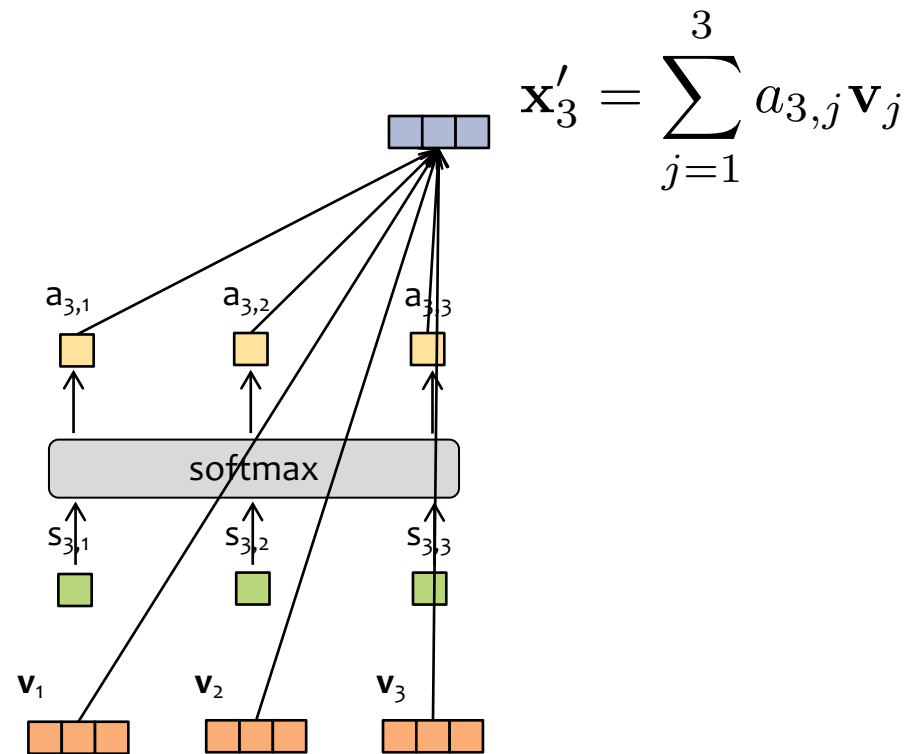
Attention



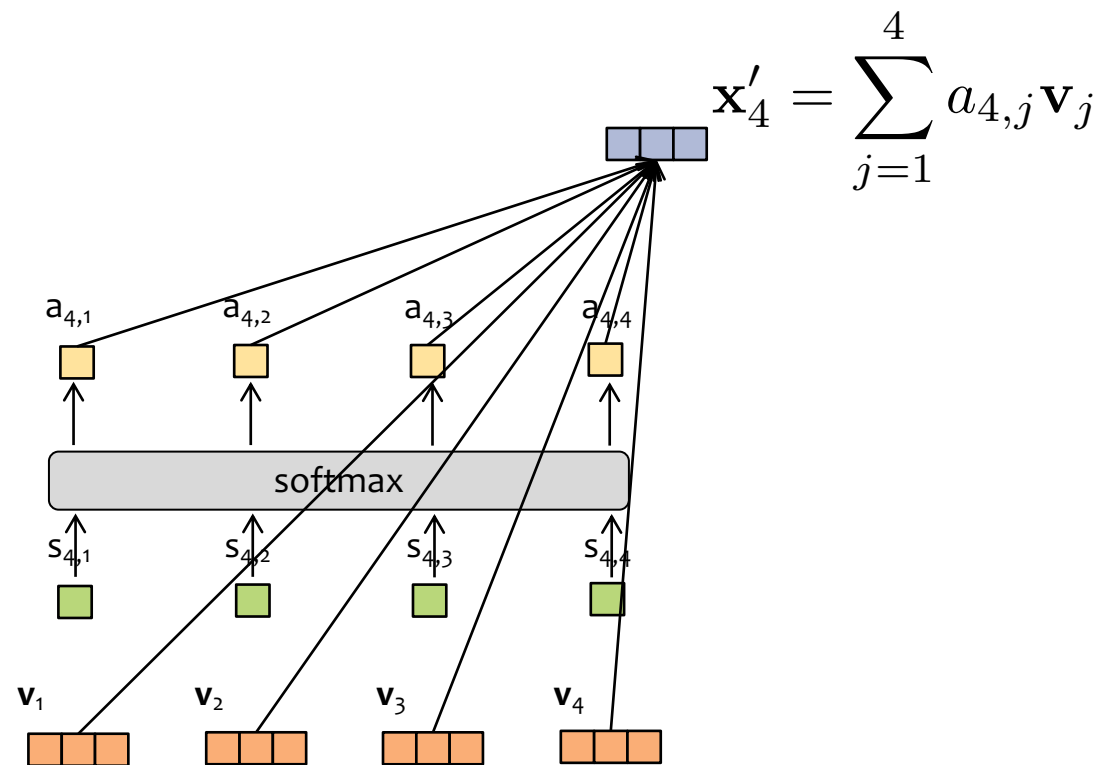
Attention



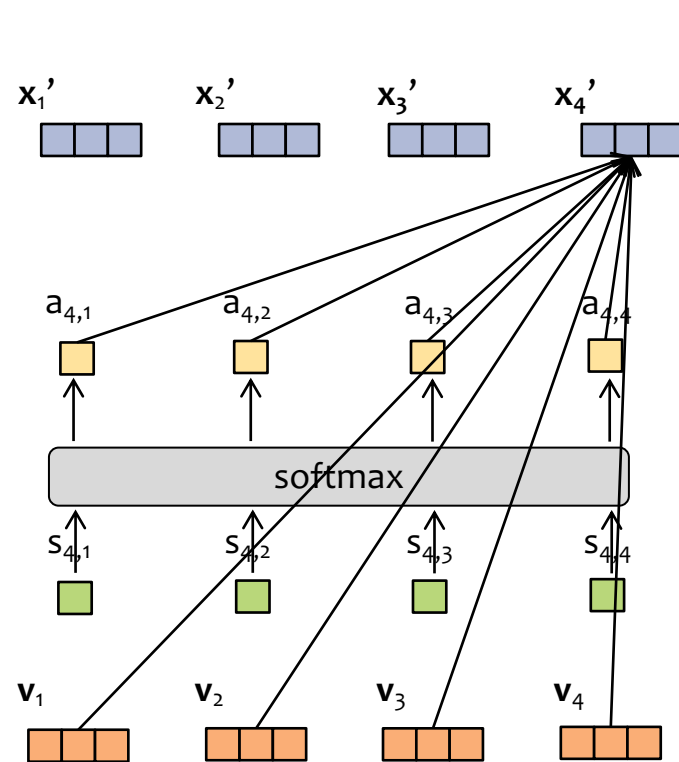
Attention



Attention



Attention



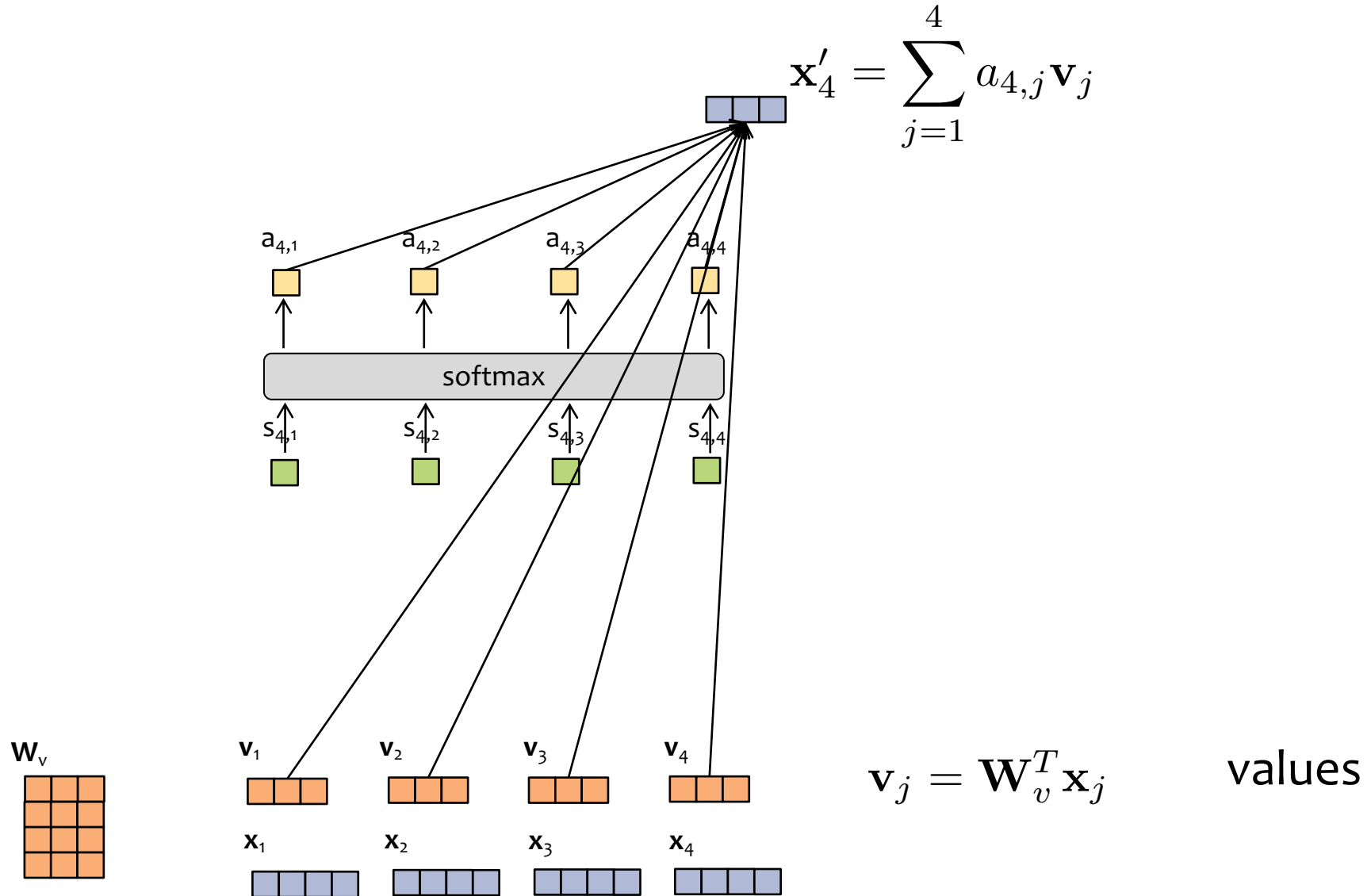
$$\mathbf{x}'_t = \sum_{j=1}^t a_{t,j} \mathbf{v}_j$$

attention weights

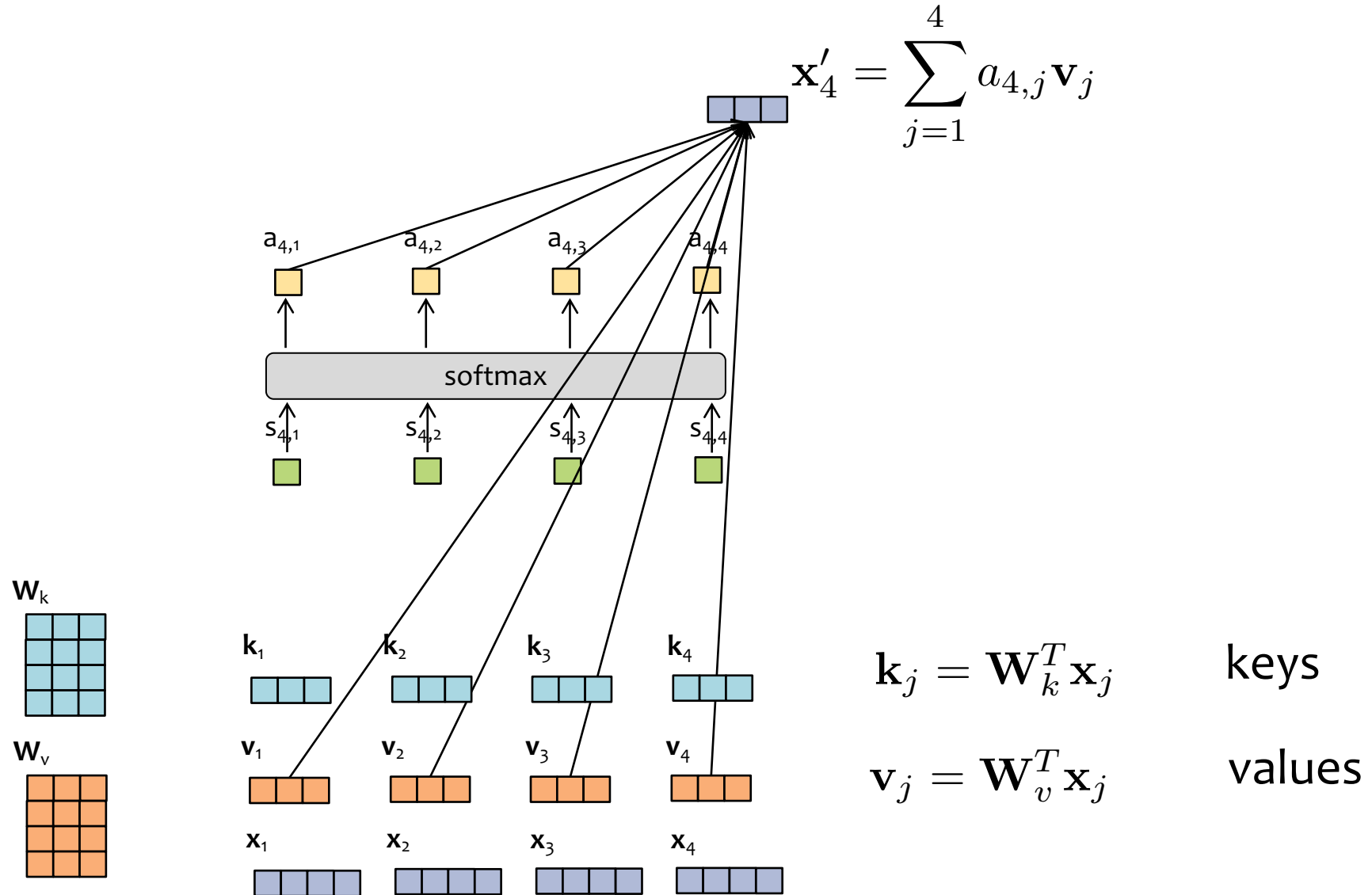
scores

values

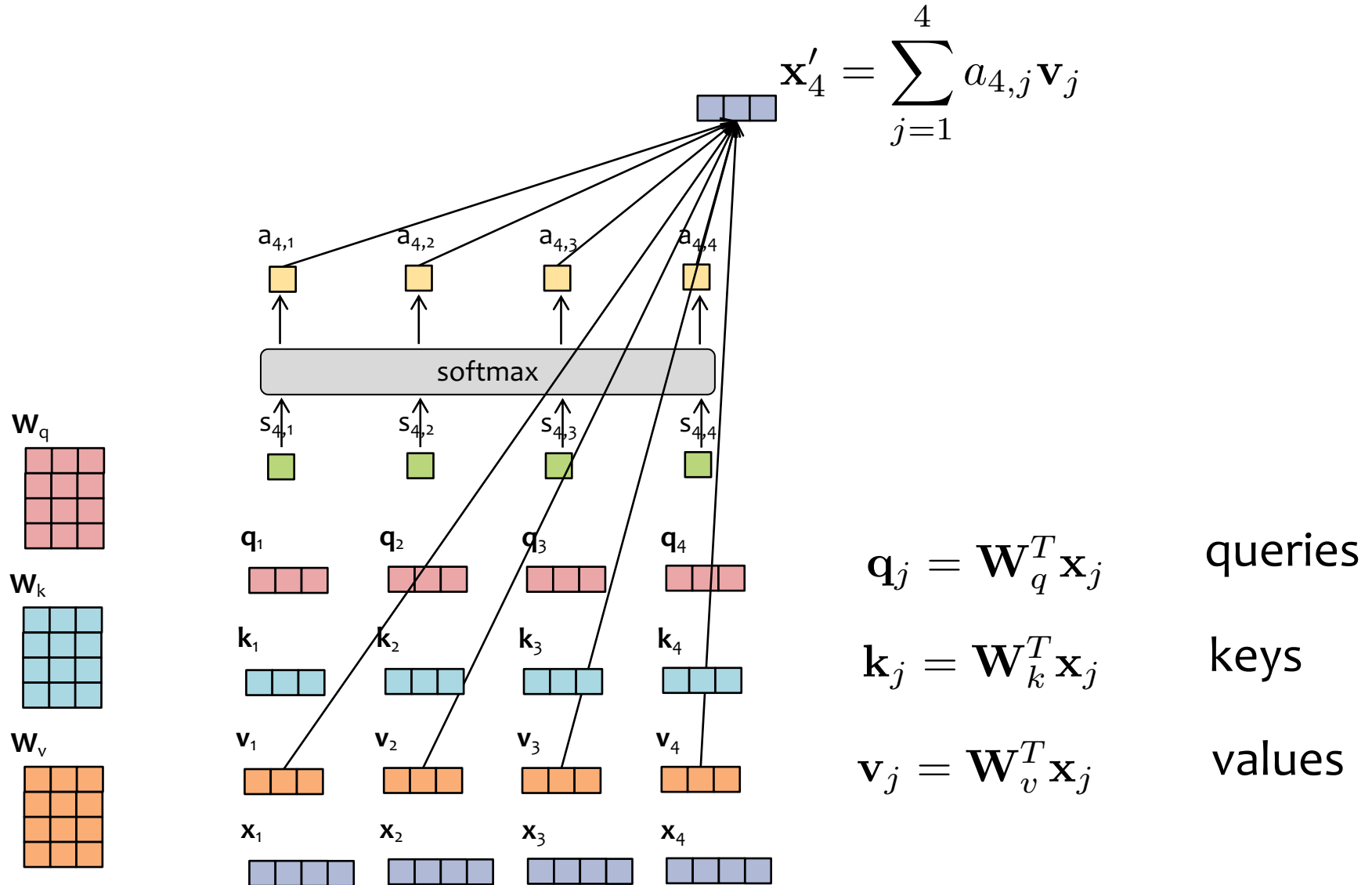
Scaled Dot-Product Attention



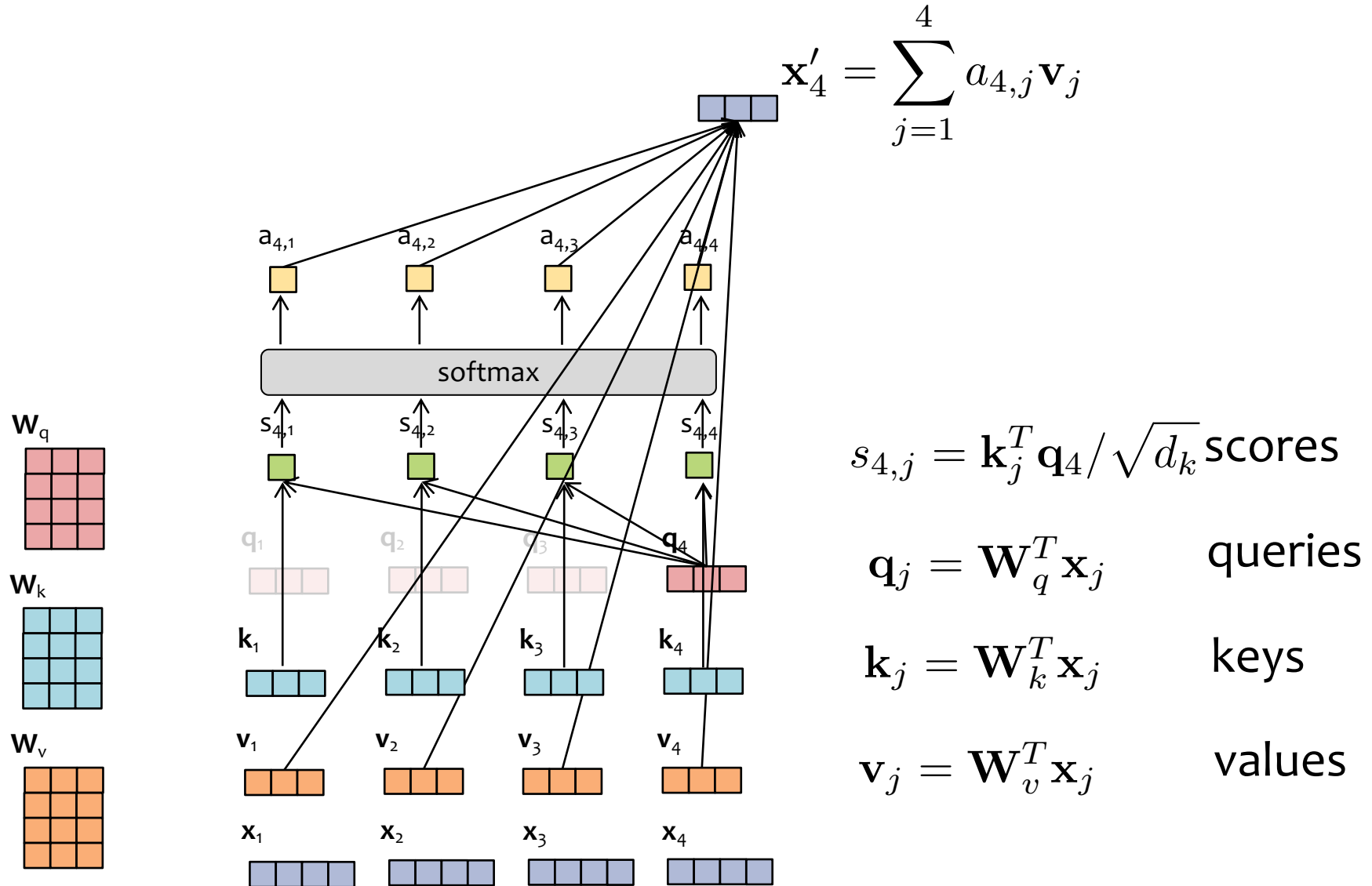
Scaled Dot-Product Attention



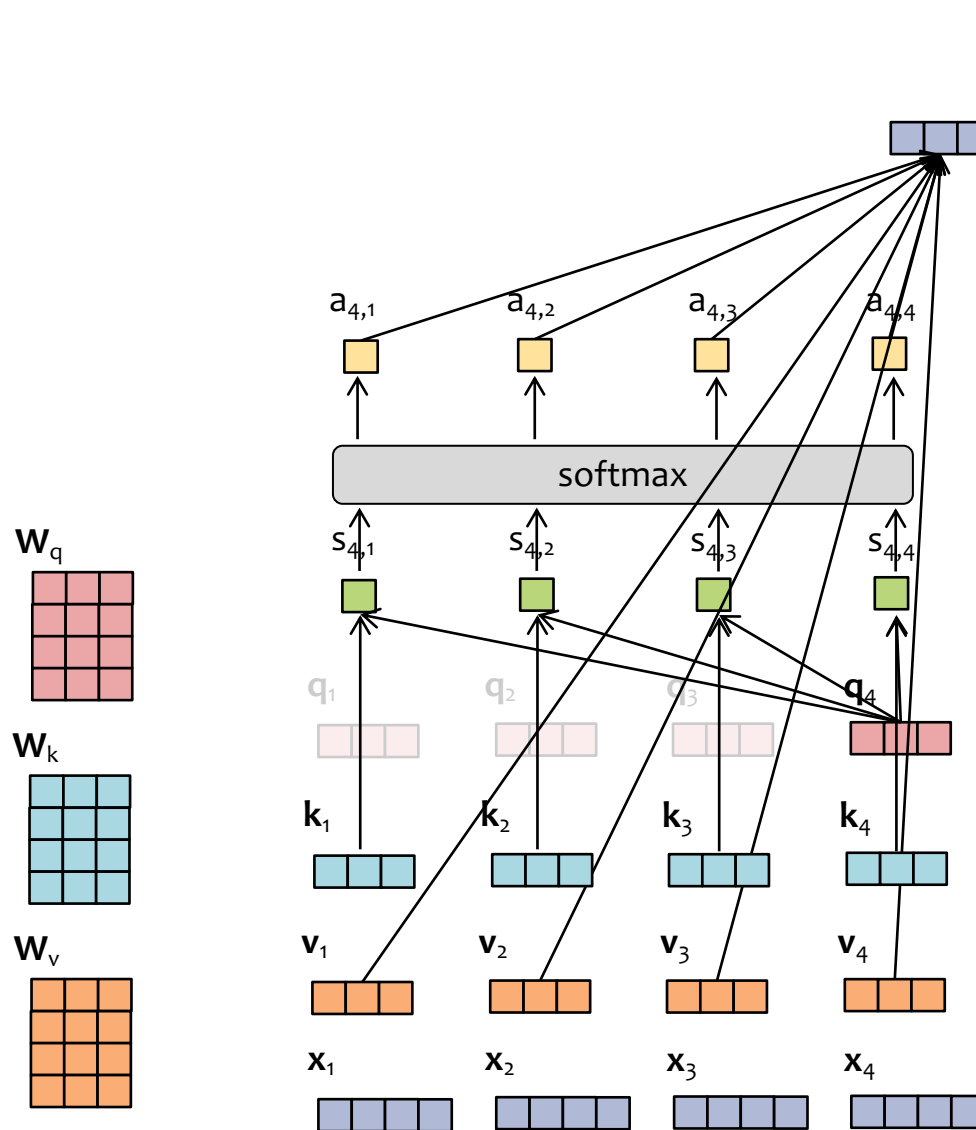
Scaled Dot-Product Attention



Scaled Dot-Product Attention



Scaled Dot-Product Attention



$$\mathbf{x}'_4 = \sum_{j=1}^4 a_{4,j} \mathbf{v}_j$$

$\mathbf{a}_4 = \text{softmax}(s_4)$ attention weights

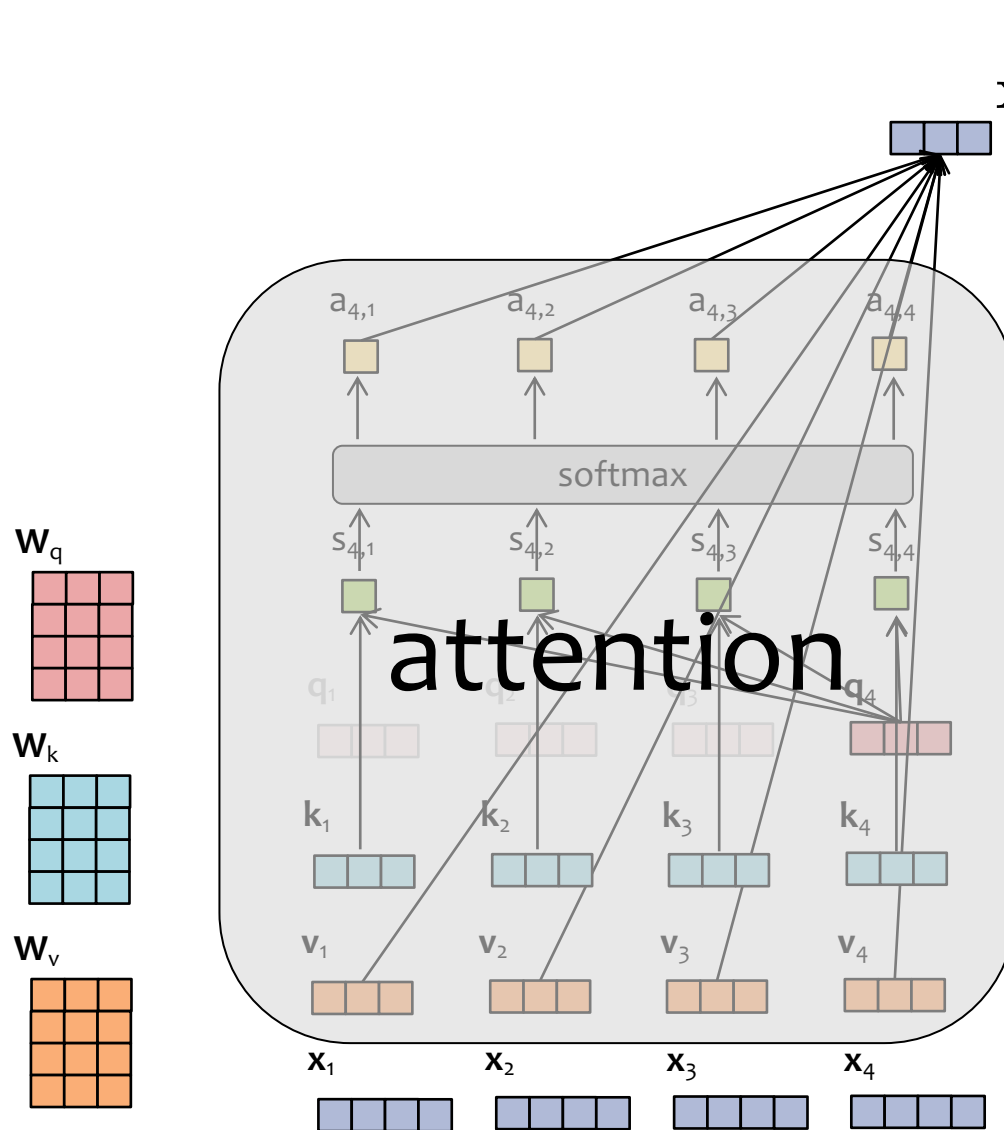
$s_{4,j} = \mathbf{k}_j^T \mathbf{q}_4 / \sqrt{d_k}$ scores

$\mathbf{q}_j = \mathbf{W}_q^T \mathbf{x}_j$ queries

$\mathbf{k}_j = \mathbf{W}_k^T \mathbf{x}_j$ keys

$\mathbf{v}_j = \mathbf{W}_v^T \mathbf{x}_j$ values

Scaled Dot-Product Attention



$$\mathbf{x}'_4 = \sum_{j=1}^4 a_{4,j} \mathbf{v}_j$$

$\mathbf{a}_4 = \text{softmax}(s_4)$ attention weights

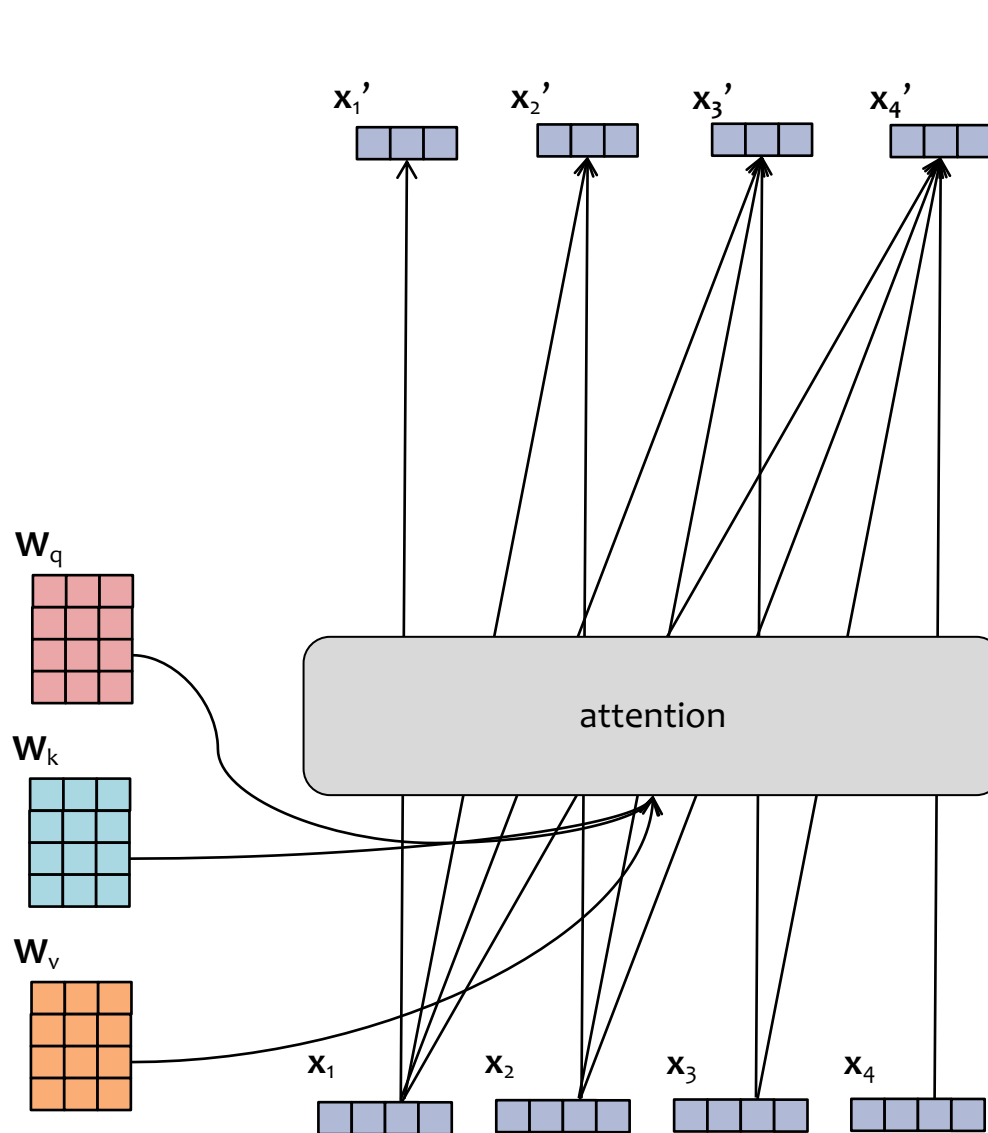
$s_{4,j} = \mathbf{k}_j^T \mathbf{q}_4 / \sqrt{d_k}$ scores

$\mathbf{q}_j = \mathbf{W}_q^T \mathbf{x}_j$ queries

$\mathbf{k}_j = \mathbf{W}_k^T \mathbf{x}_j$ keys

$\mathbf{v}_j = \mathbf{W}_v^T \mathbf{x}_j$ values

Scaled Dot-Product Attention



$$\mathbf{x}'_t = \sum_{j=1}^t a_{t,j} \mathbf{v}_j$$

$\mathbf{a}_t = \text{softmax}(\mathbf{s}_t)$ attention weights

$$s_{t,j} = \mathbf{k}_j^T \mathbf{q}_t / \sqrt{d_k} \text{ scores}$$

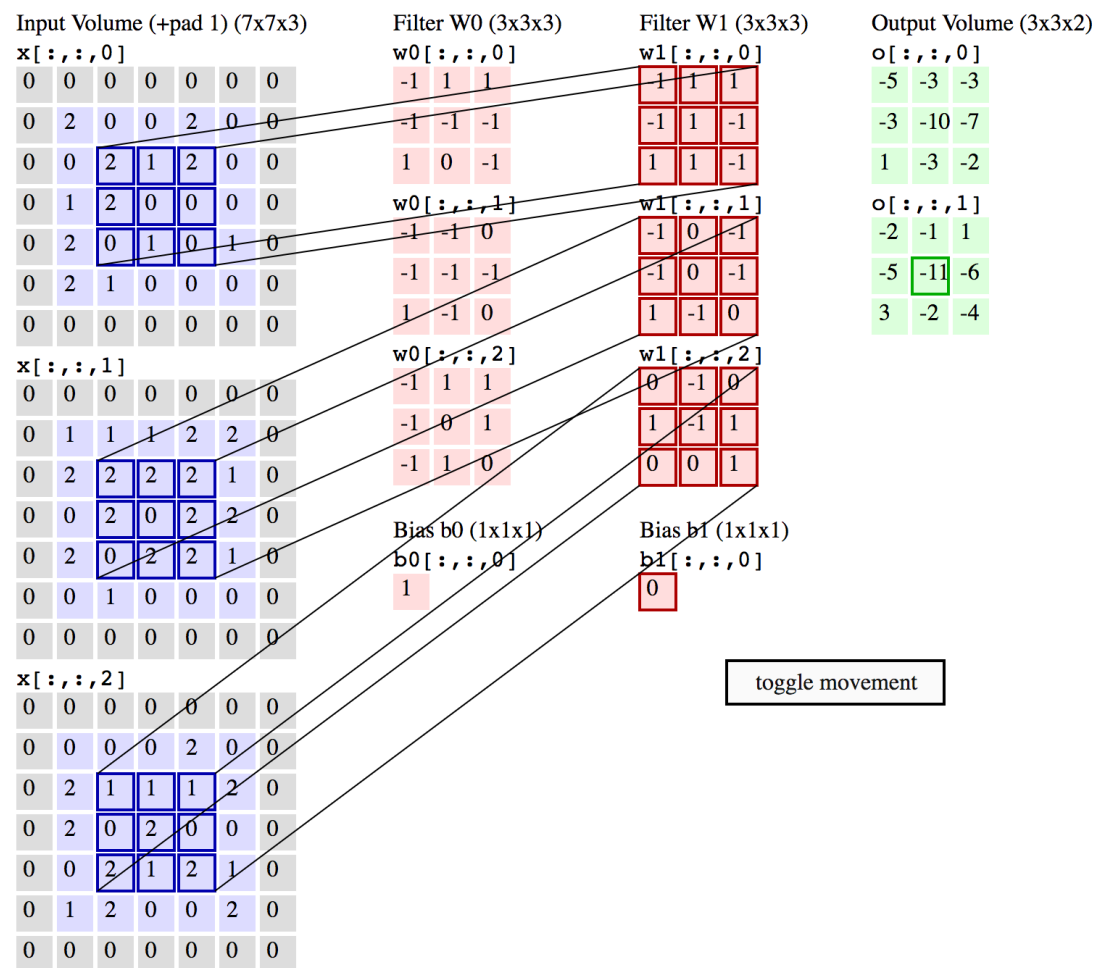
$$\mathbf{q}_j = \mathbf{W}_q^T \mathbf{x}_j \quad \text{queries}$$

$$\mathbf{k}_j = \mathbf{W}_k^T \mathbf{x}_j \quad \text{keys}$$

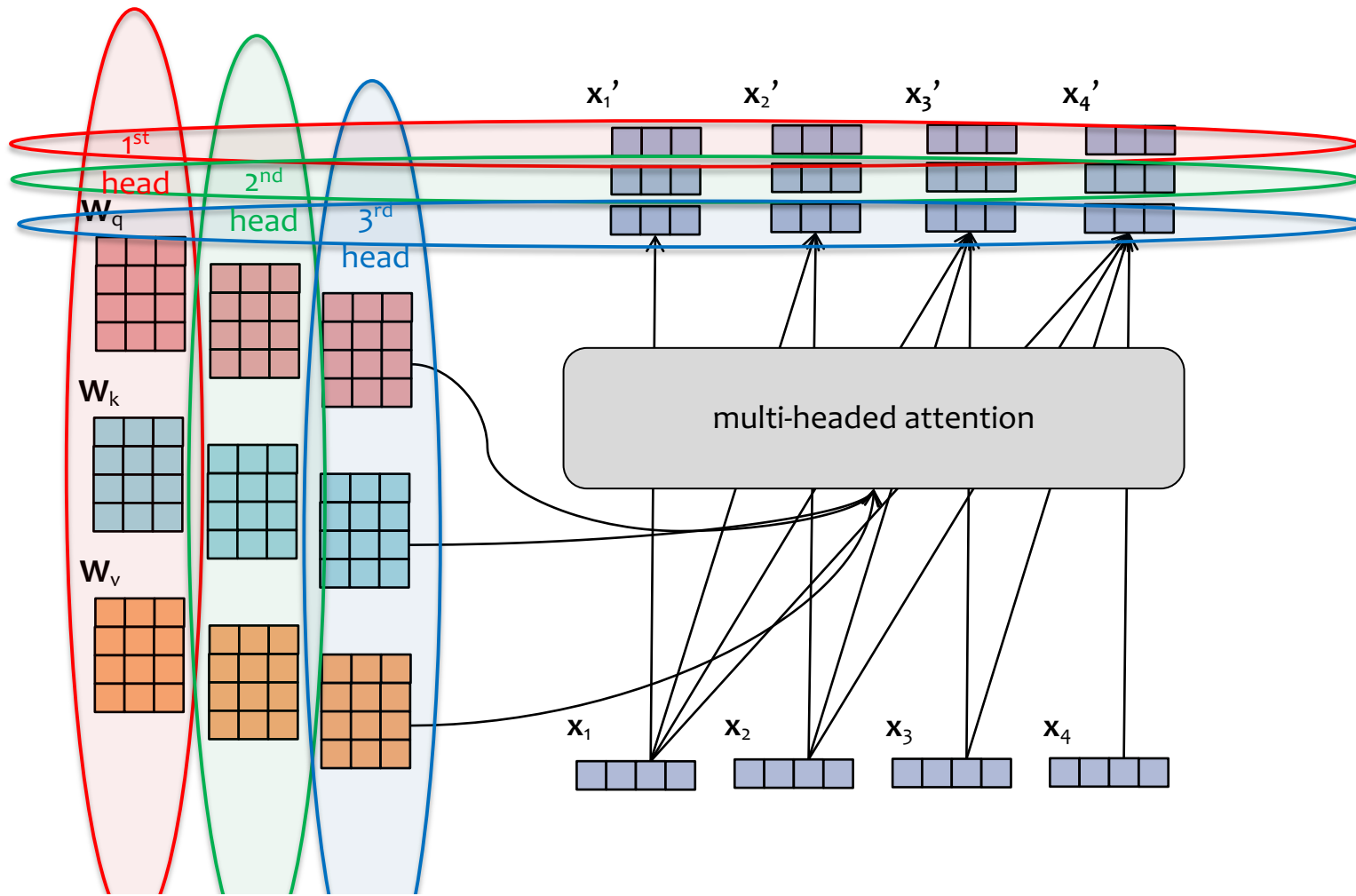
$$\mathbf{v}_j = \mathbf{W}_v^T \mathbf{x}_j \quad \text{values}$$

Animation of 3D Convolution

<http://cs231n.github.io/convolutional-networks/>



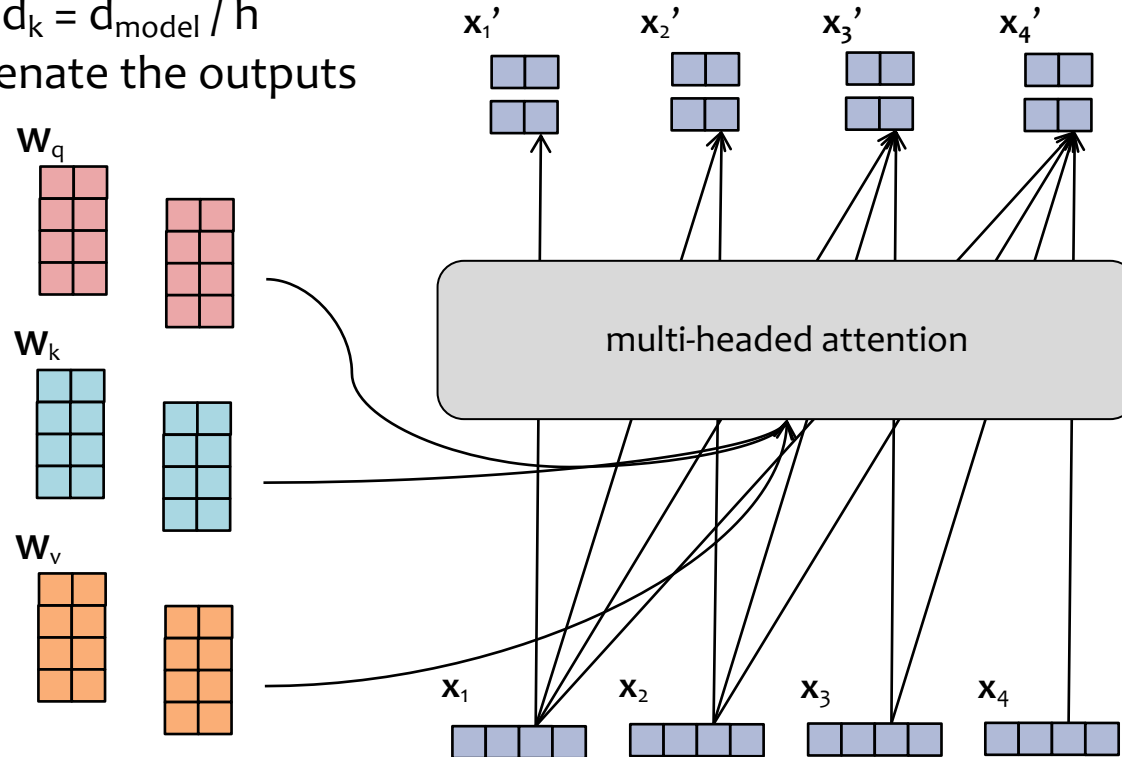
Multi-headed Attention



- Just as we can have **multiple channels** in a **convolution** layer, we can use **multiple heads** in an **attention** layer
- Each head gets **its own parameters**
- We can **concatenate** all the outputs to get a single vector for each time step

Multi-headed Attention

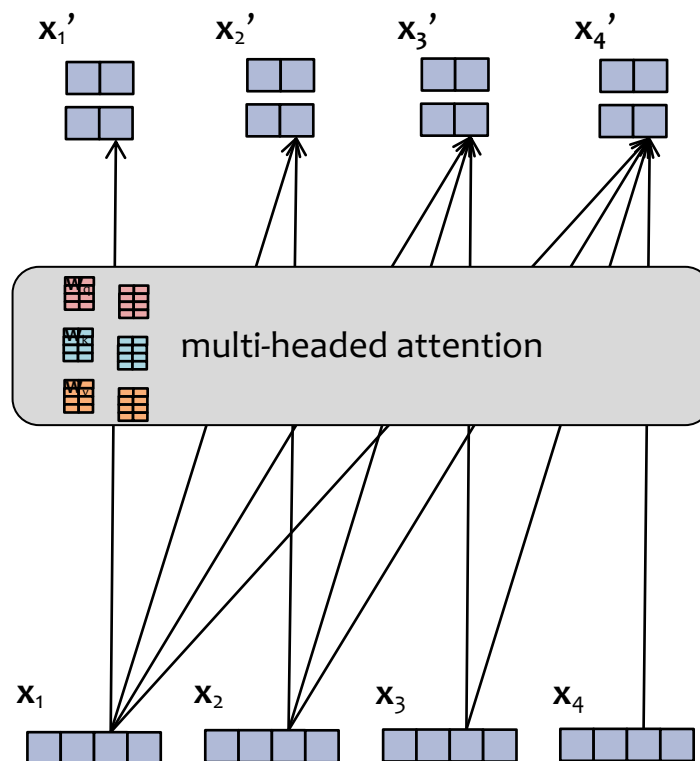
- To ensure the dimension of the **input** embedding $\mathbf{x}_t$ is the same as the **output** embedding $\mathbf{x}_t'$, Transformers usually choose the embedding sizes and number of heads appropriately:
 - $d_{\text{model}} = \text{dim. of inputs}$
 - $d_k = \text{dim. of each output}$
 - $h = \# \text{ of heads}$
 - Choose $d_k = d_{\text{model}} / h$
- Then concatenate the outputs



- Just as we can have **multiple channels** in a **convolution** layer, we can use **multiple heads** in an **attention** layer
- Each head gets **its own parameters**
- We can **concatenate** all the outputs to get a single vector for each time step

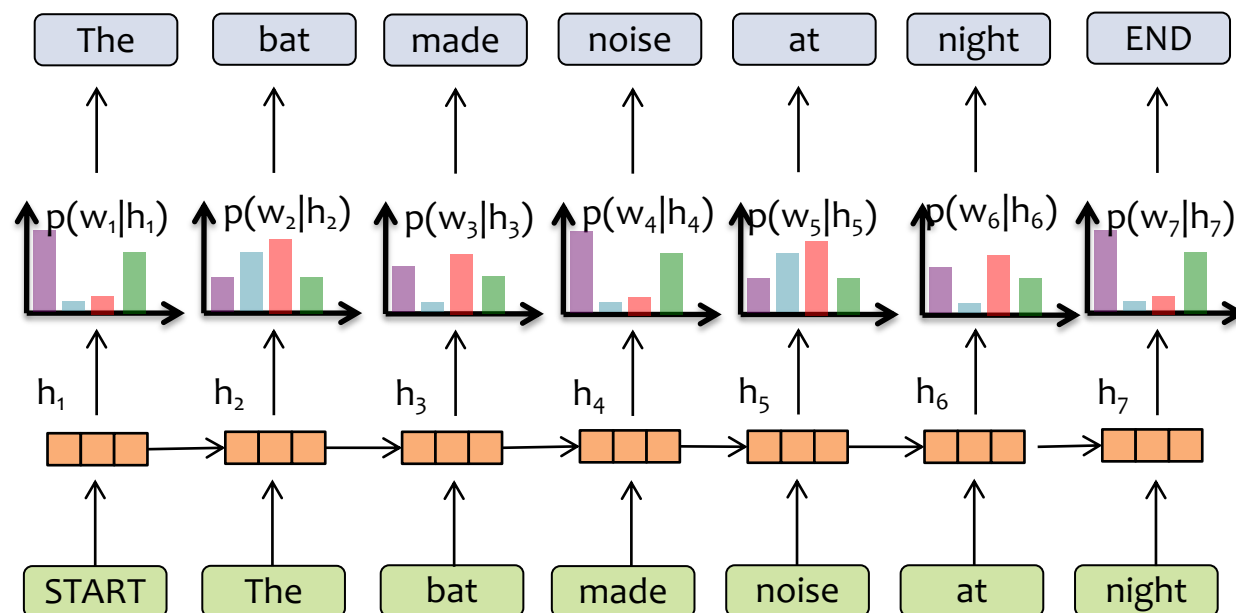
- To ensure the dimension of the **input** embedding $\mathbf{x}_t$ is the same as the **output** embedding $\mathbf{x}_t'$, Transformers usually choose the embedding sizes and number of heads appropriately:
 - $d_{\text{model}} = \text{dim. of inputs}$
 - $d_k = \text{dim. of each output}$
 - $h = \# \text{ of heads}$
 - Choose $d_k = d_{\text{model}} / h$
- Then concatenate the outputs

Multi-headed Attention



- Just as we can have **multiple channels** in a **convolution** layer, we can use **multiple heads** in an **attention** layer
- Each head gets **its own parameters**
- We can **concatenate** all the outputs to get a single vector for each time step

RNN Language Model



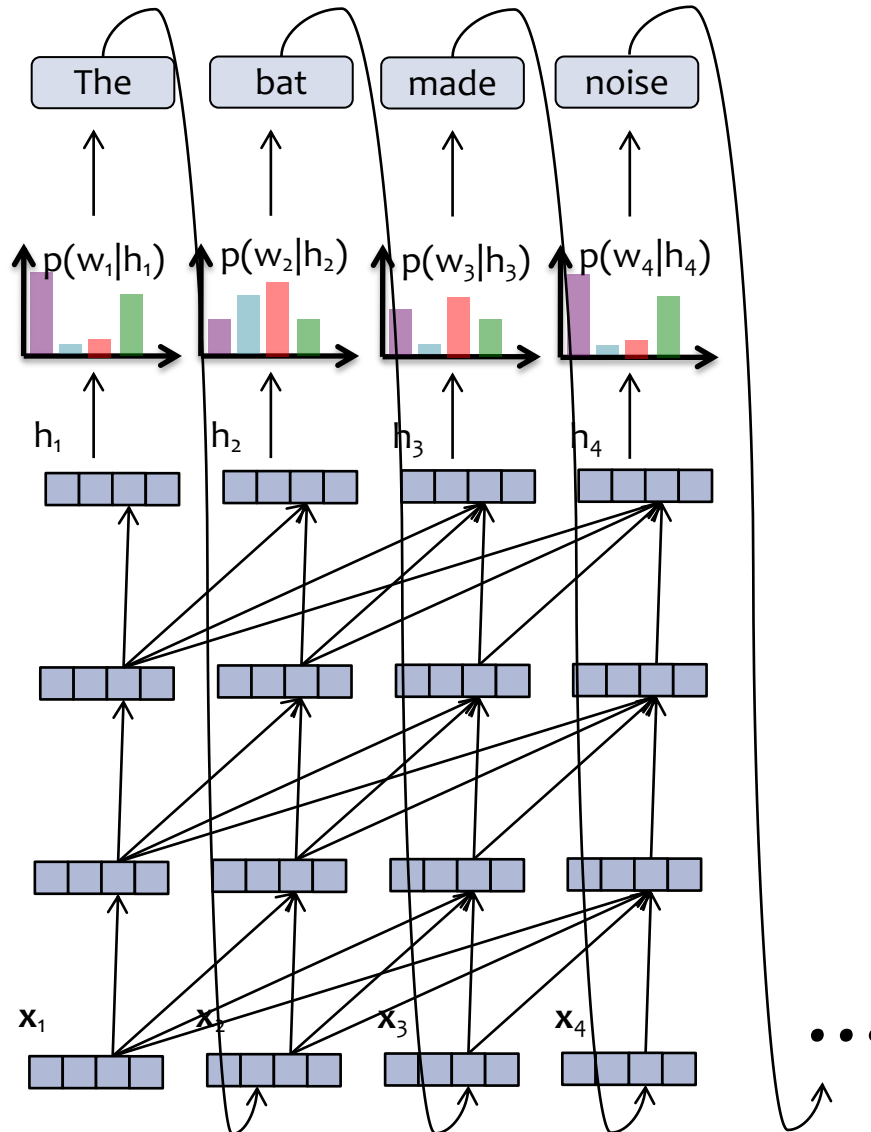
Key Idea:

- (1) convert all previous words to a **fixed length vector**
- (2) define distribution $p(w_t | f_{\theta}(w_{t-1}, \dots, w_1))$ that conditions on the vector $\mathbf{h}_t = f_{\theta}(w_{t-1}, \dots, w_1)$

Transformer Language Model

Important!

- RNN computation graph grows **linearly** with the number of input tokens
- Transformer-LM computation graph grows **quadratically** with the number of input tokens



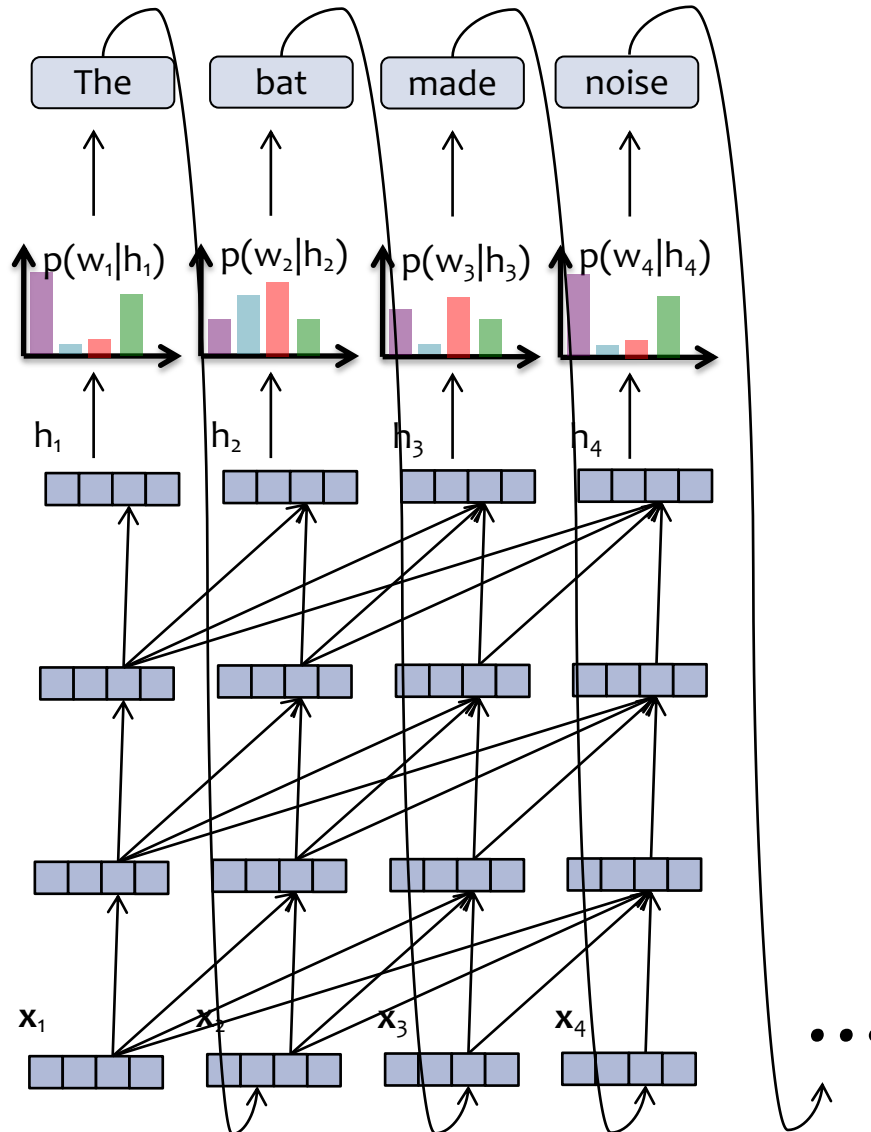
Each hidden vector looks back at the hidden vectors of the **current and previous timesteps in the previous layer**.

The language model part is just like an RNN-LM!

Transformer Language Model

Important!

- RNN computation graph grows **linearly** with the number of input tokens
- Transformer-LM computation graph grows **quadratically** with the number of input tokens



Each layer of a Transformer LM consists of several **sublayers**:

1. attention
2. feed-forward neural network
3. layer normalization
4. residual connections

Each hidden vector looks back at the hidden vectors of the **current and previous timesteps in the previous layer**.

The language model part is just like an RNN-LM!

Layer Normalization

- *The Problem:* **internal covariate shift** occurs during training of a deep network when a small change in the low layers amplifies into a large change in the high layers
- *One Solution:* **Layer normalization** normalizes each layer and learns elementwise gain/bias
- Such normalization allows for higher learning rates (for **faster convergence**) without issues of diverging gradients

Given input $\mathbf{a} \in \mathbb{R}^K$, LayerNorm computes output $\mathbf{b} \in \mathbb{R}^K$:

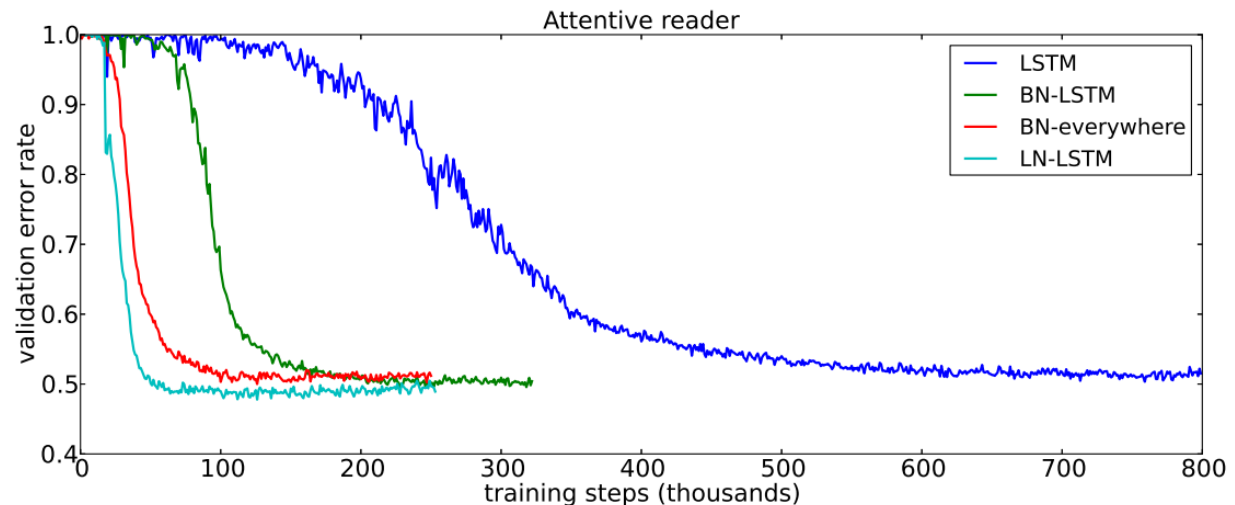
$$\mathbf{b} = \gamma \odot \frac{\mathbf{a} - \mu}{\sigma} \oplus \beta$$

where we have mean $\mu = \frac{1}{K} \sum_{k=1}^K a_k$,

standard deviation $\sigma = \sqrt{\frac{1}{K} \sum_{k=1}^K (a_k - \mu)^2}$,

and parameters $\gamma \in \mathbb{R}^K, \beta \in \mathbb{R}^K$.

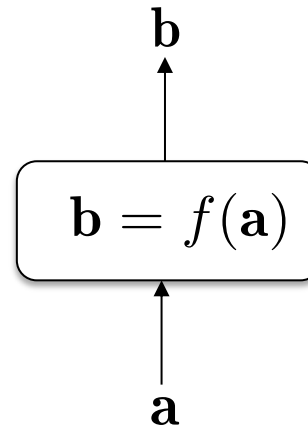
$\odot$ and $\oplus$ denote elementwise multiplication and addition.



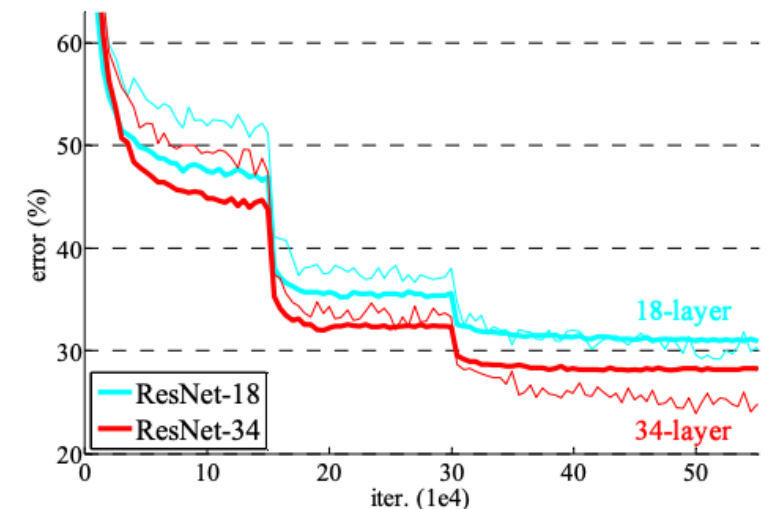
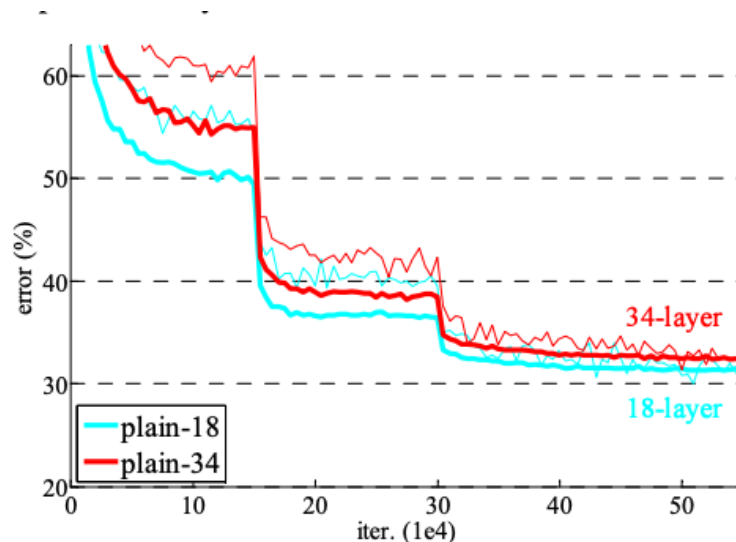
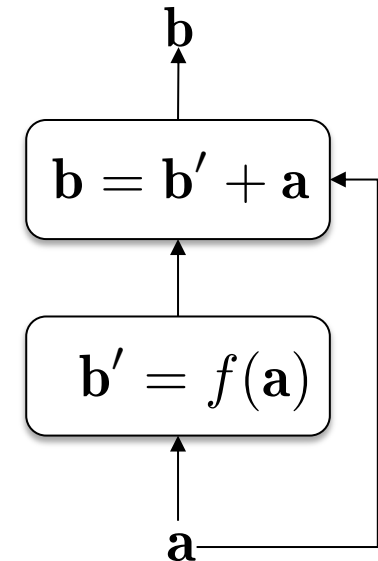
Residual Connections

- **The Problem:** as network depth grows very large, a **performance degradation** occurs that is not explained by overfitting (i.e. train / test error both worsen)
- **One Solution: Residual connections** pass a copy of the input alongside another function so that information can flow more directly
- These residual connections allow for **effective training of very deep networks** that perform better than their shallower (though still deep) counterparts

Plain Connection



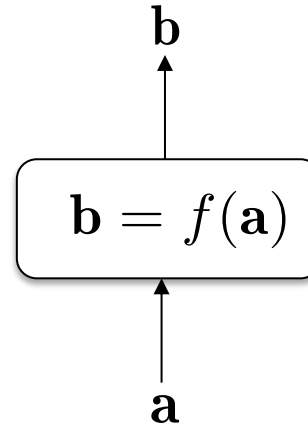
Residual Connection



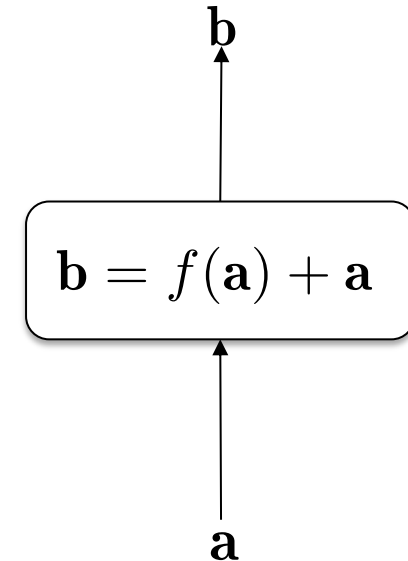
Residual Connections

- **The Problem:** as network depth grows very large, a **performance degradation** occurs that is not explained by overfitting (i.e. train / test error both worsen)
- **One Solution: Residual connections** pass a copy of the input alongside another function so that information can flow more directly
- These residual connections allow for **effective training of very deep networks** that perform better than their shallower (though still deep) counterparts

Plain Connection



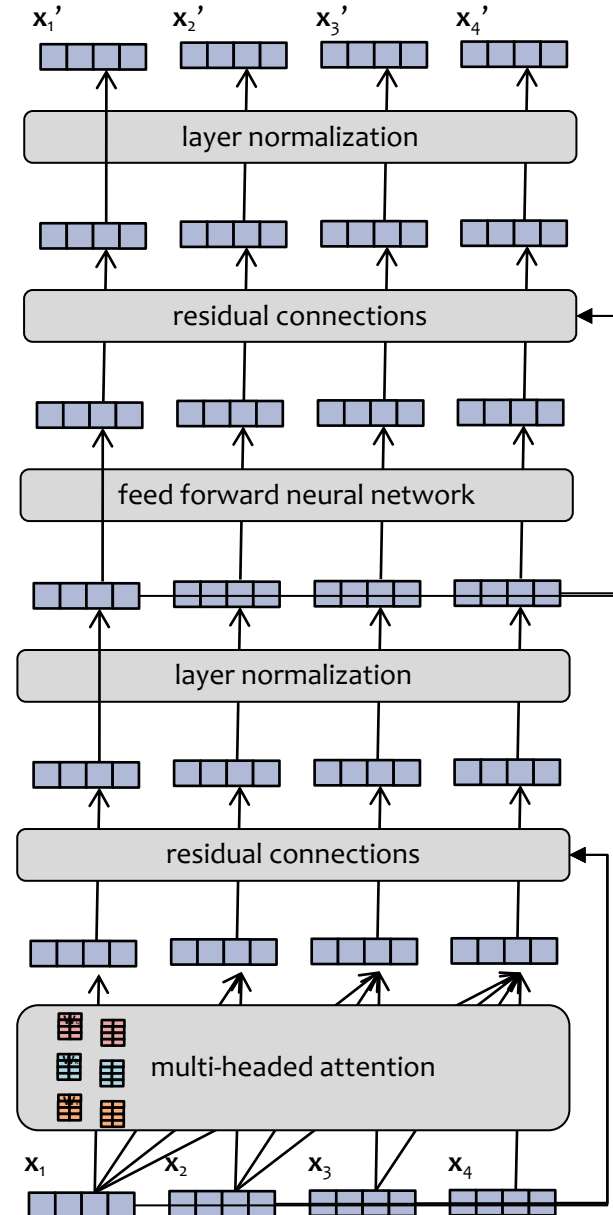
Residual Connection



Why are residual connections helpful?

Instead of $f(a)$ having to learn a full transformation of a , $f(a)$ only needs to learn an additive modification of a (i.e. the residual).

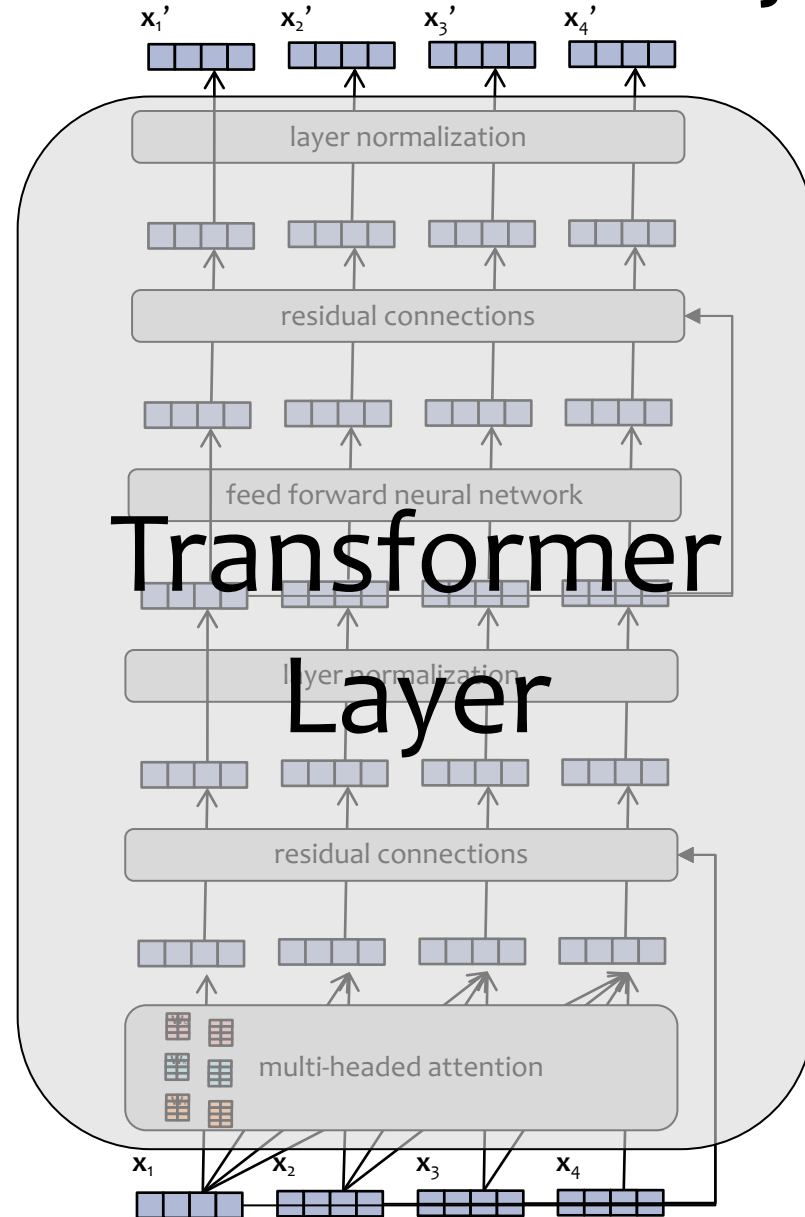
Transformer Layer



Each layer of a Transformer LM consists of several **sublayers**:

1. attention
2. feed-forward neural network
3. layer normalization
4. residual connections

Transformer Layer



Each layer of a Transformer LM consists of several **sublayers**:

1. attention
2. feed-forward neural network
3. layer normalization
4. residual connections

Transformer Layer



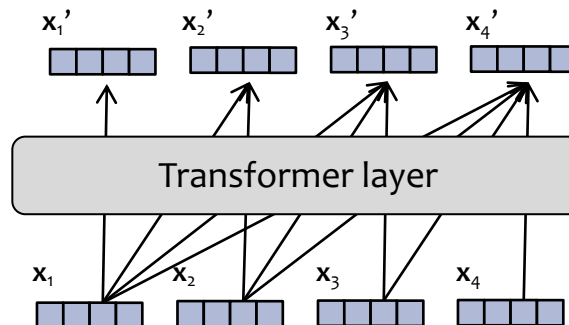
Each layer of a Transformer LM consists of several **sublayers**:

1. attention
2. feed-forward neural network
3. layer normalization
4. residual connections

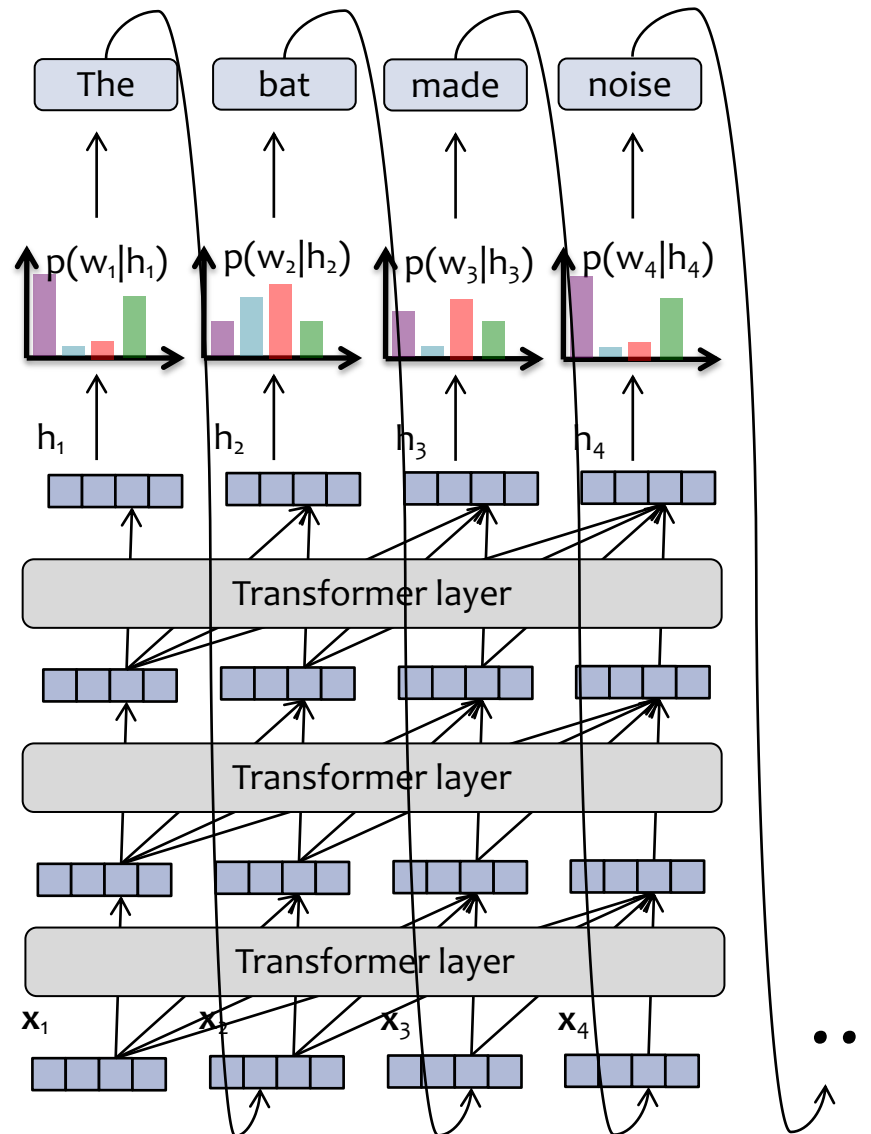
Transformer Layer

Each layer of a Transformer LM consists of several **sublayers**:

1. attention
2. feed-forward neural network
3. layer normalization
4. residual connections



Transformer Language Model



Each layer of a Transformer LM consists of several **sublayers**:

1. attention
2. feed-forward neural network
3. layer normalization
4. residual connections

Each hidden vector looks back at the hidden vectors of the **current and previous timesteps in the previous layer**.

The language model part is just like an RNN-LM.

In-Class Exercise

Question:

Suppose we have the following input embeddings and attention weights:

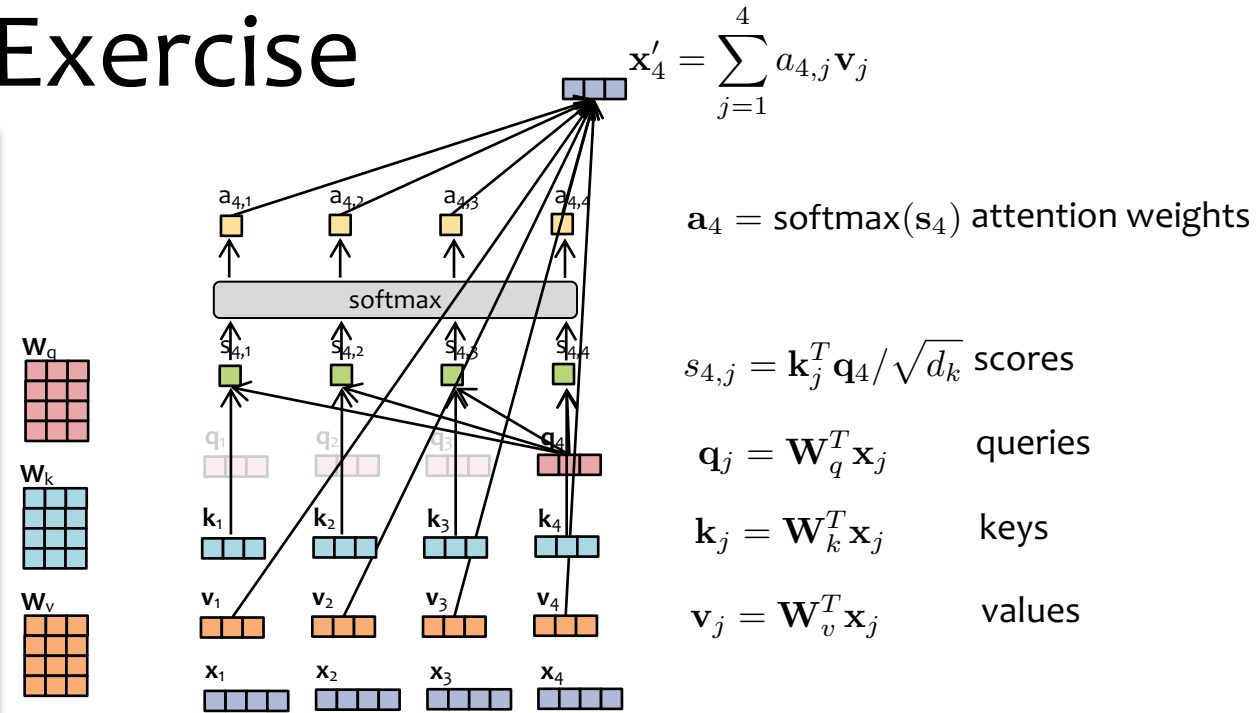
- $x_1 = [1, 0, 0, 0]$ $a_{4,1} = 0.1$
- $x_2 = [0, 1, 0, 0]$ $a_{4,2} = 0.2$
- $x_3 = [0, 0, 2, 0]$ $a_{4,3} = 0.6$
- $x_4 = [0, 0, 0, 1]$ $a_{4,4} = 0.1$

And $W_v = I$. Then we can compute x_4' .

Now suppose we swap the embeddings x_2 and x_3 such that

- $x_2 = [0, 0, 2, 0]$
- $x_3 = [0, 1, 0, 0]$

What is the new value of x_4' ?



Answer:

In-Class Exercise

Question:

Suppose we have the following input embeddings and attention weights:

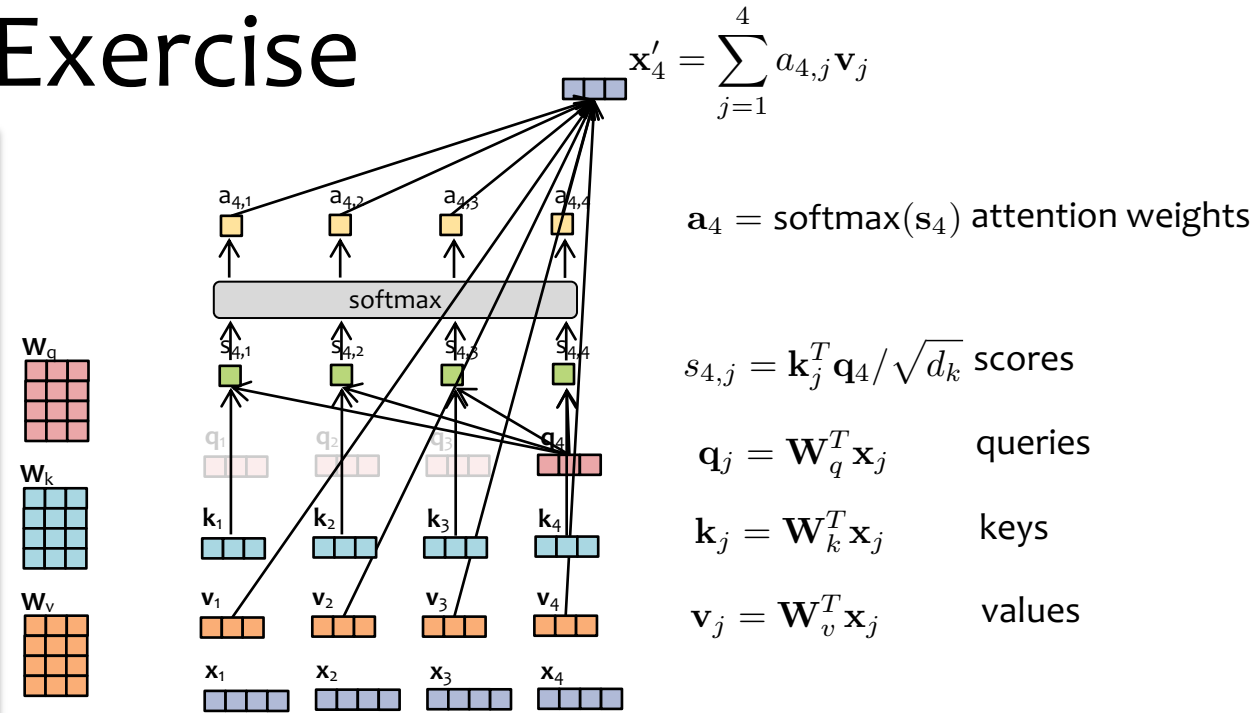
- $x_1 = [1, 0, 0, 0]$ $a_{4,1} = 0.1$
- $x_2 = [0, 1, 0, 0]$ $a_{4,2} = 0.2$
- $x_3 = [0, 0, 2, 0]$ $a_{4,3} = 0.6$
- $x_4 = [0, 0, 0, 1]$ $a_{4,4} = 0.1$

And $W_v = I$. Then we can compute x_4' .

Now suppose we swap the embeddings x_2 and x_3 such that

- $x_2 = [0, 0, 2, 0]$
- $x_3 = [0, 1, 0, 0]$

What is the new value of x_4' ?

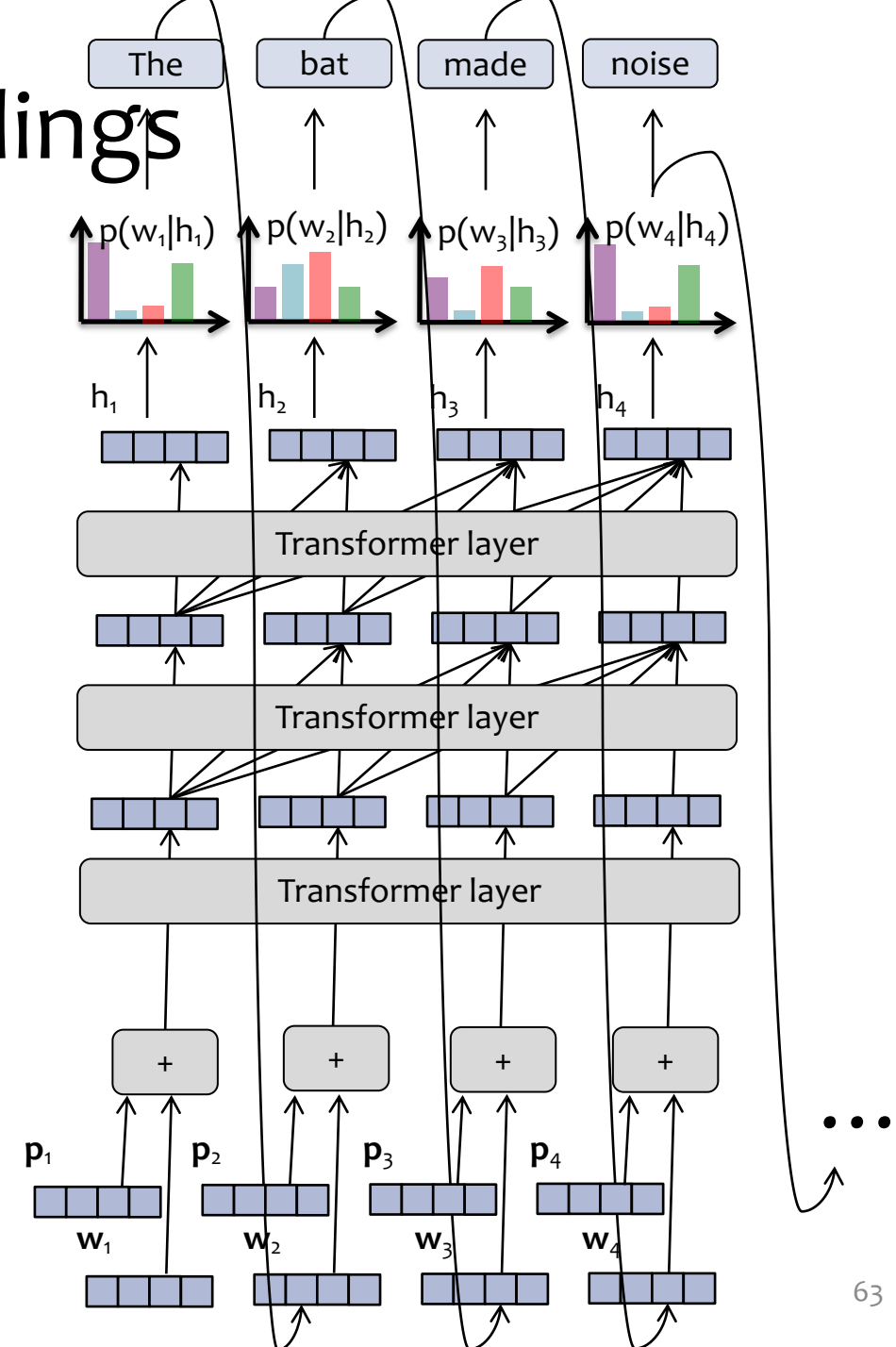


Answer:

Exactly the same as it was before!

Position Embeddings

- **The Problem:** Because attention is position invariant, we **need** a way to learn about positions
- **The Solution:** Use (or learn) a collection of position specific embeddings: $\mathbf{p}_t$ represents what it means to be in position t . And add this to the word embedding $\mathbf{w}_t$. The **key idea** is that every word that appears in position t uses the same position embedding $\mathbf{p}_t$
- There are a number of varieties of position embeddings:
 - Some are fixed (based on sine and cosine), whereas others are learned (like word embeddings)
 - Some are absolute (as described above) but we can also use relative position embeddings (i.e. relative to the position of the query vector)



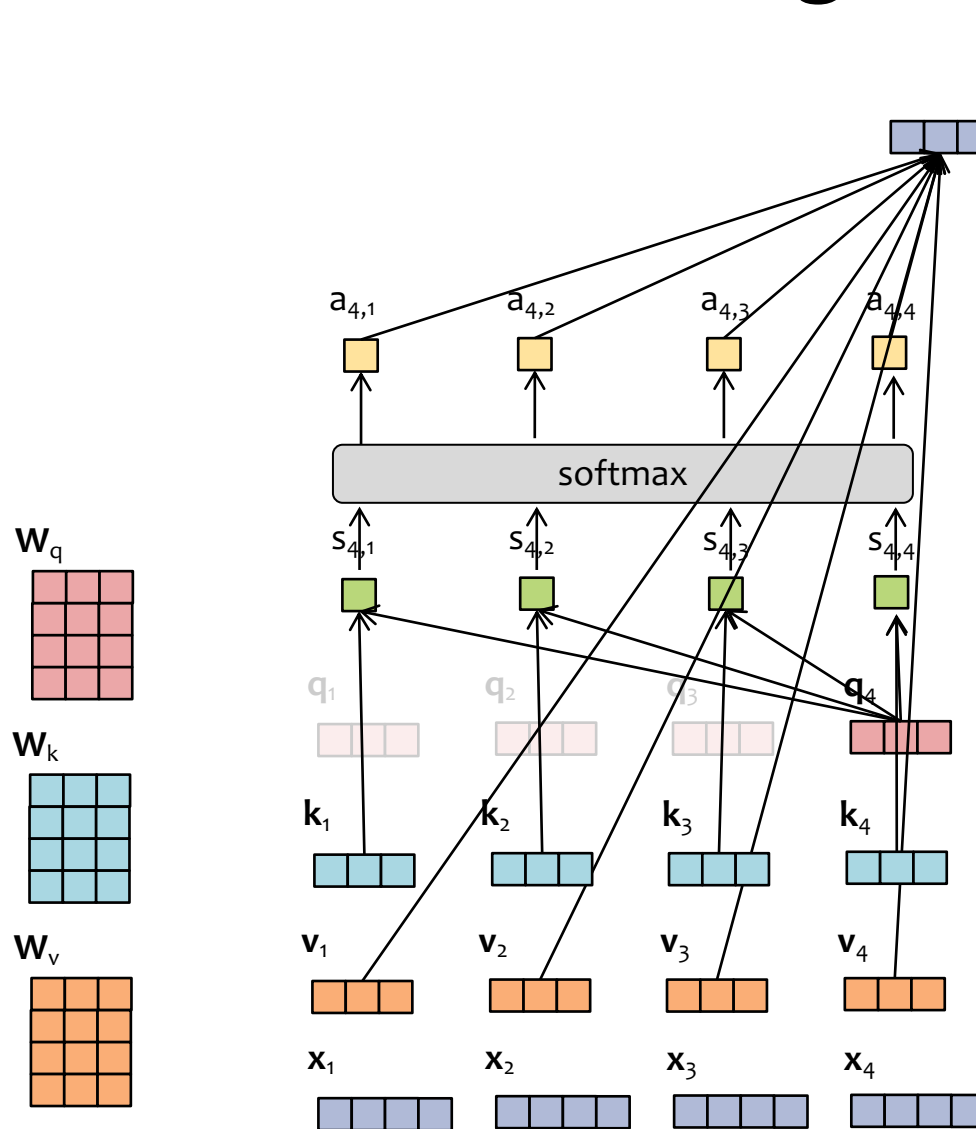
GPT-3

- GPT stands for Generative Pre-trained Transformer
- GPT is just a Transformer LM, but with a huge number of parameters

Model	# layers	dimension of states	dimension of inner states	# attention heads	# params
GPT (2018)	12	768	4*768	12	117M
GPT-2 (2019)	48	1600	4*1600	12	1542M
GPT-3 (2020)	96	12288	4*12288	96	175000M
GPT-4o (2024)	?	?	?	?	? 1-2 trillion
GPT-5 (2025)	?	?	?	?	? 1-2 trillion

IMPLEMENTING A TRANSFORMER LM

Matrix Version of Single-Headed Attention



$$\mathbf{x}'_4 = \sum_{j=1}^4 a_{4,j} \mathbf{v}_j$$

$\mathbf{a}_4 = \text{softmax}(\mathbf{s}_4)$ attention weights

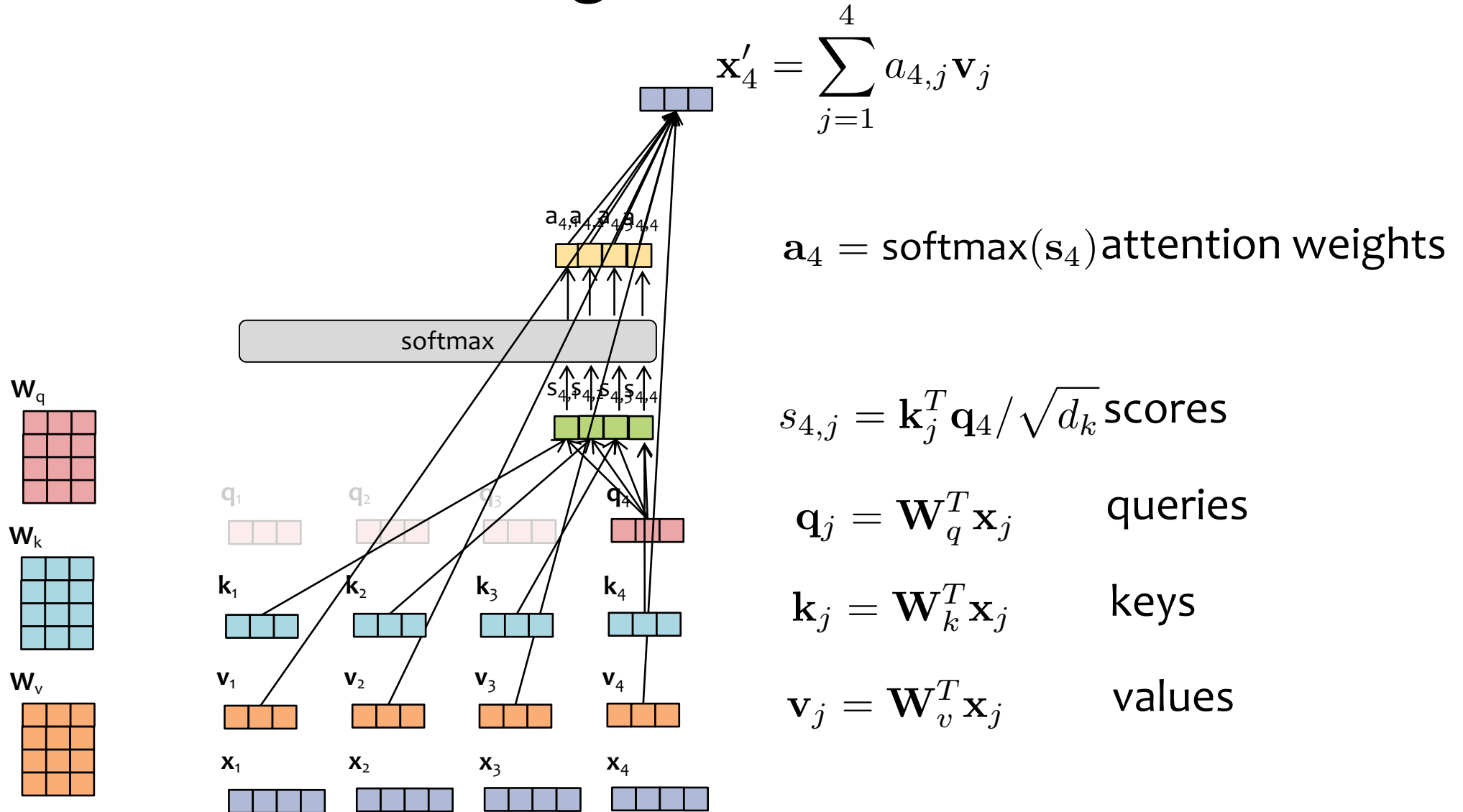
$s_{4,j} = \mathbf{k}_j^T \mathbf{q}_4 / \sqrt{d_k}$ scores

$\mathbf{q}_j = \mathbf{W}_q^T \mathbf{x}_j$ queries

$\mathbf{k}_j = \mathbf{W}_k^T \mathbf{x}_j$ keys

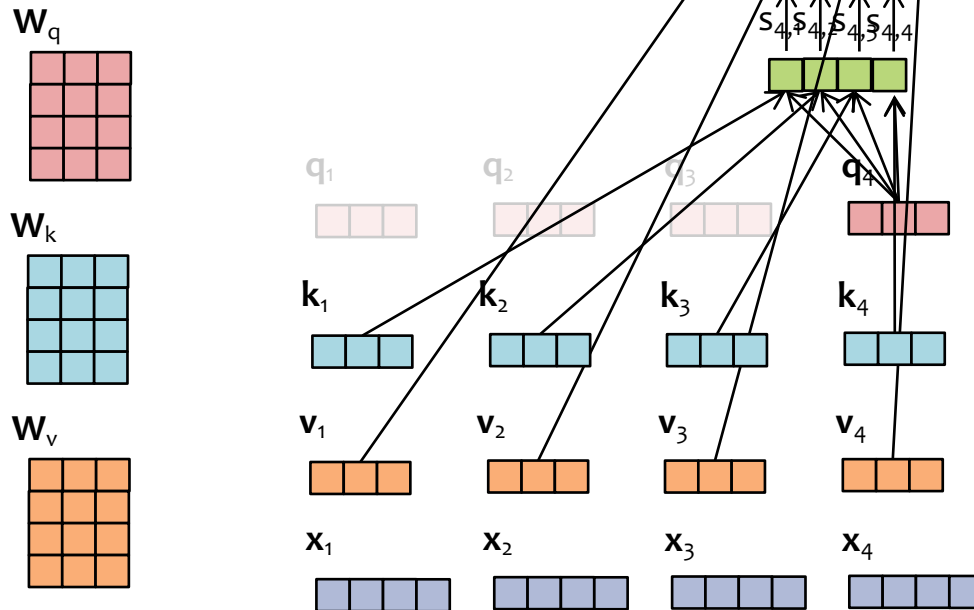
$\mathbf{v}_j = \mathbf{W}_v^T \mathbf{x}_j$ values

Matrix Version of Single-Headed Attention



Matrix Version of Single-Headed Attention

- For speed, we compute all the queries at once using matrix operations
- First we pack the queries, keys, values into matrices
- Then we compute all the queries at once



$$\mathbf{X}' = \mathbf{A}\mathbf{V} = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k})\mathbf{V}$$

$$\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_4]^T = \text{softmax}(\mathbf{S})$$

$$\mathbf{S} = [\mathbf{s}_1, \dots, \mathbf{s}_4]^T = \mathbf{Q}\mathbf{K}^T / \sqrt{d_k}$$

$$\mathbf{Q} = [\mathbf{q}_1, \dots, \mathbf{q}_4]^T = \mathbf{X}\mathbf{W}_q$$

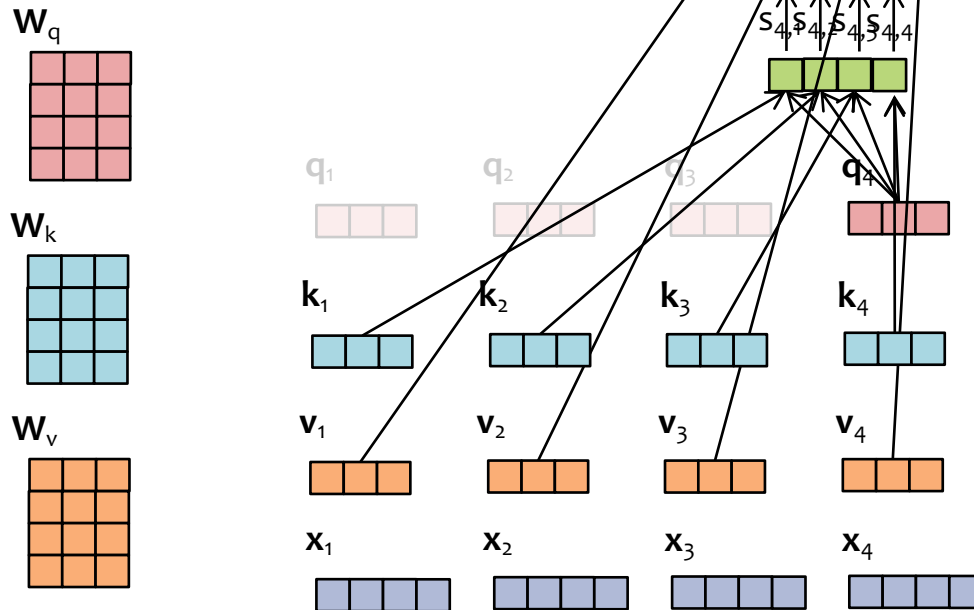
$$\mathbf{K} = [\mathbf{k}_1, \dots, \mathbf{k}_4]^T = \mathbf{X}\mathbf{W}_k$$

$$\mathbf{V} = [\mathbf{v}_1, \dots, \mathbf{v}_4]^T = \mathbf{X}\mathbf{W}_v$$

$$\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_4]^T$$

Matrix Version of Single-Headed Attention

- For speed, we compute all the queries at once using matrix operations
- First we pack the queries, keys, values into matrices
- Then we compute all the queries at once



$$\mathbf{X}' = \mathbf{A}\mathbf{V} = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k})\mathbf{V}$$

$$\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_4]^T = \text{softmax}(\mathbf{S})$$

$$\mathbf{S} = [\mathbf{s}_1, \dots, \mathbf{s}_4]^T = \mathbf{Q}\mathbf{K}^T / \sqrt{d_k}$$

$$\mathbf{Q} = [\mathbf{q}_1, \dots, \mathbf{q}_4]^T = \mathbf{X}\mathbf{W}_q \in \mathbb{R}^{N \times d_k}$$

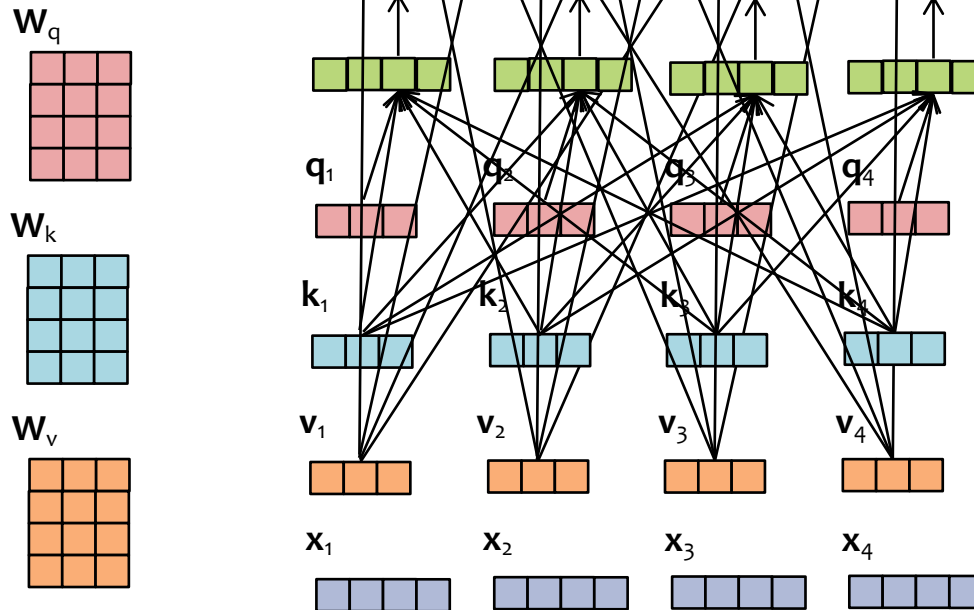
$$\mathbf{K} = [\mathbf{k}_1, \dots, \mathbf{k}_4]^T = \mathbf{X}\mathbf{W}_k \in \mathbb{R}^{N \times d_k}$$

$$\mathbf{V} = [\mathbf{v}_1, \dots, \mathbf{v}_4]^T = \mathbf{X}\mathbf{W}_v \in \mathbb{R}^{N \times d_k}$$

$$\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_4]^T \in \mathbb{R}^{N \times d_{\text{model}}}$$

Matrix Version of Single-Headed Attention

- For speed, we compute all the queries at once using matrix operations
- First we pack the queries, keys, values into matrices
- Then we compute all the queries at once



$$\mathbf{X}' = \mathbf{A}\mathbf{V} = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k})\mathbf{V} \\ \in \mathbb{R}^{N \times d_k}$$

$$\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_4]^T = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}$$

$$\mathbf{S} = [\mathbf{s}_1, \dots, \mathbf{s}_4]^T = \mathbf{Q}\mathbf{K}^T / \sqrt{d_k} \in \mathbb{R}^{N \times N}$$

$$\mathbf{Q} = [\mathbf{q}_1, \dots, \mathbf{q}_4]^T = \mathbf{X}\mathbf{W}_q \in \mathbb{R}^{N \times d_k}$$

$$\mathbf{K} = [\mathbf{k}_1, \dots, \mathbf{k}_4]^T = \mathbf{X}\mathbf{W}_k \in \mathbb{R}^{N \times d_k}$$

$$\mathbf{V} = [\mathbf{v}_1, \dots, \mathbf{v}_4]^T = \mathbf{X}\mathbf{W}_v \in \mathbb{R}^{N \times d_k}$$

$$\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_4]^T \in \mathbb{R}^{N \times d_{\text{model}}}$$

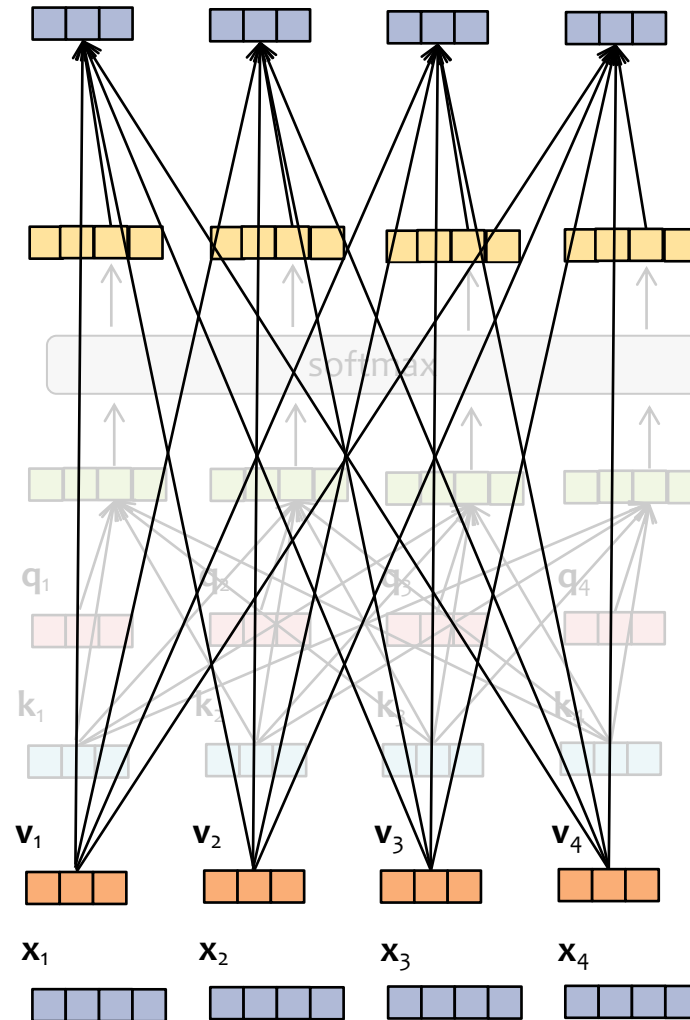
Matrix Version of Single-Headed Attention

Holy cow, that's a lot of new arrows... do we always want/need all of those?

- Suppose we're training our transformer to predict the next token(s) given the input...
- ... then attending to tokens that come after the current token is cheating!

So what is this model?

- This version is the *standard* Transformer block. (more on this later!)
- But we want the Transformer LM block
- And that requires masking!



$$\mathbf{X}' = \mathbf{A}\mathbf{V} = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k})\mathbf{V}$$

$$\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_4]^T = \text{softmax}(\mathbf{S})$$

$$\mathbf{S} = [\mathbf{s}_1, \dots, \mathbf{s}_4]^T = \mathbf{Q}\mathbf{K}^T / \sqrt{d_k}$$

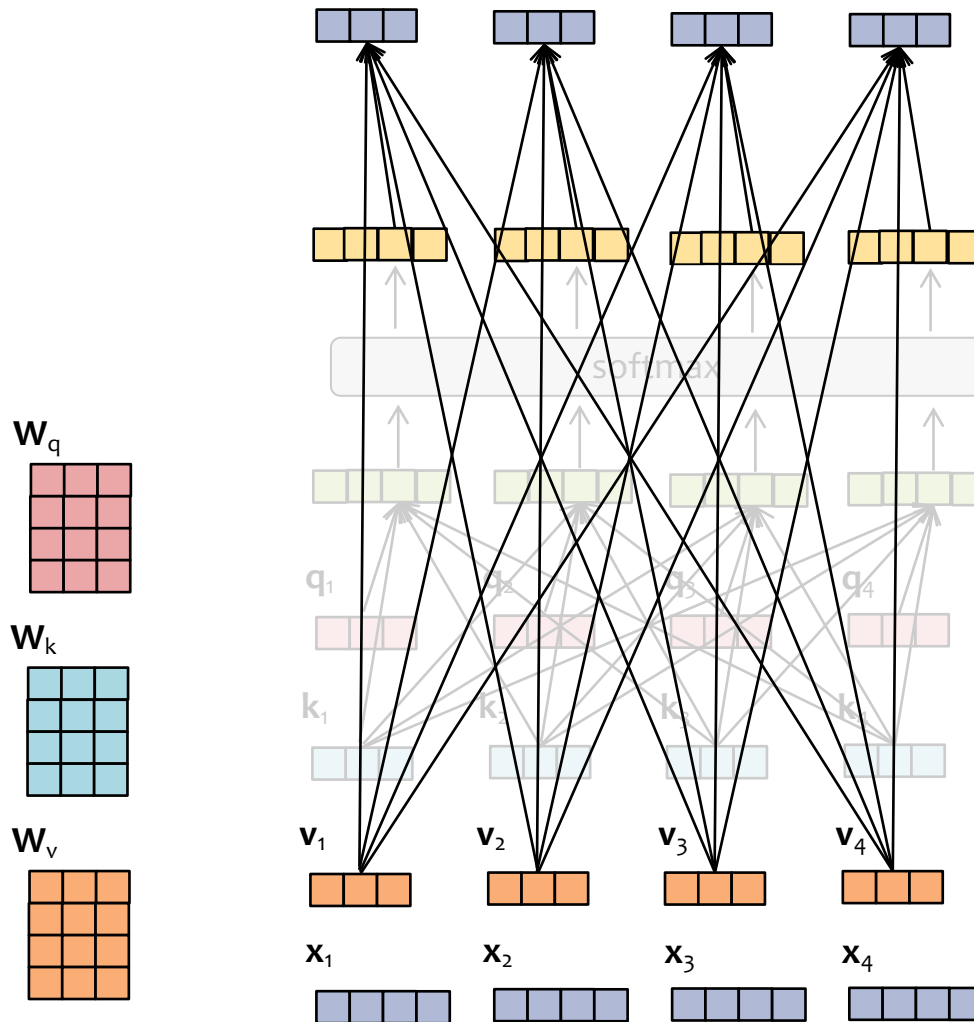
$$\mathbf{Q} = [\mathbf{q}_1, \dots, \mathbf{q}_4]^T = \mathbf{X}\mathbf{W}_q$$

$$\mathbf{K} = [\mathbf{k}_1, \dots, \mathbf{k}_4]^T = \mathbf{X}\mathbf{W}_k$$

$$\mathbf{V} = [\mathbf{v}_1, \dots, \mathbf{v}_4]^T = \mathbf{X}\mathbf{W}_v$$

$$\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_4]^T$$

Matrix Version of Single-Headed Attention



$$\mathbf{X}' = \mathbf{A}\mathbf{V} = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k})\mathbf{V}$$

$$\mathbf{A} = \text{softmax}(\mathbf{S})$$

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^T / \sqrt{d_k}$$

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_q$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}_k$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}_v$$

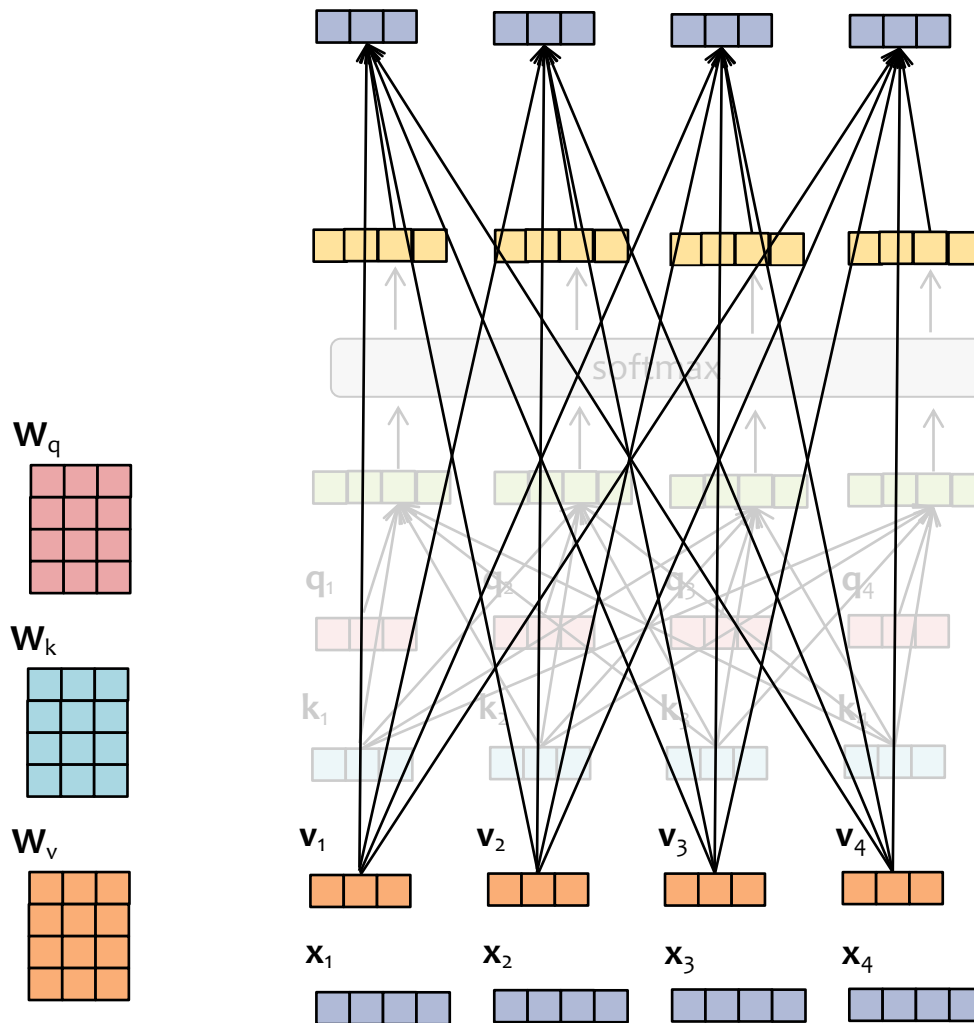
$$\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_4]^T$$

Question: How is the softmax applied?

- A. column-wise
- B. row-wise

Answer:

Matrix Version of Single-Headed Attention



$$\mathbf{X}' = \mathbf{A}\mathbf{V} = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k})\mathbf{V}$$

$$\mathbf{A} = \text{softmax}(\mathbf{S})$$

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^T / \sqrt{d_k}$$

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_q$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}_k$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}_v$$

$$\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_4]^T$$

Question: How is the softmax applied?

- A. column-wise
- B. row-wise

Answer:

Row-wise

Matrix Version of Single-Headed (Causal) Attention

Insight: if some element in the input to the softmax is $-\infty$, then the corresponding output is 0!

Question: For a causal LM which is the correct matrix?

A:

$$M = \begin{bmatrix} 0 & 0 & 0 & 0 \\ -\infty & 0 & 0 & 0 \\ -\infty & -\infty & 0 & 0 \\ -\infty & -\infty & -\infty & 0 \end{bmatrix}$$

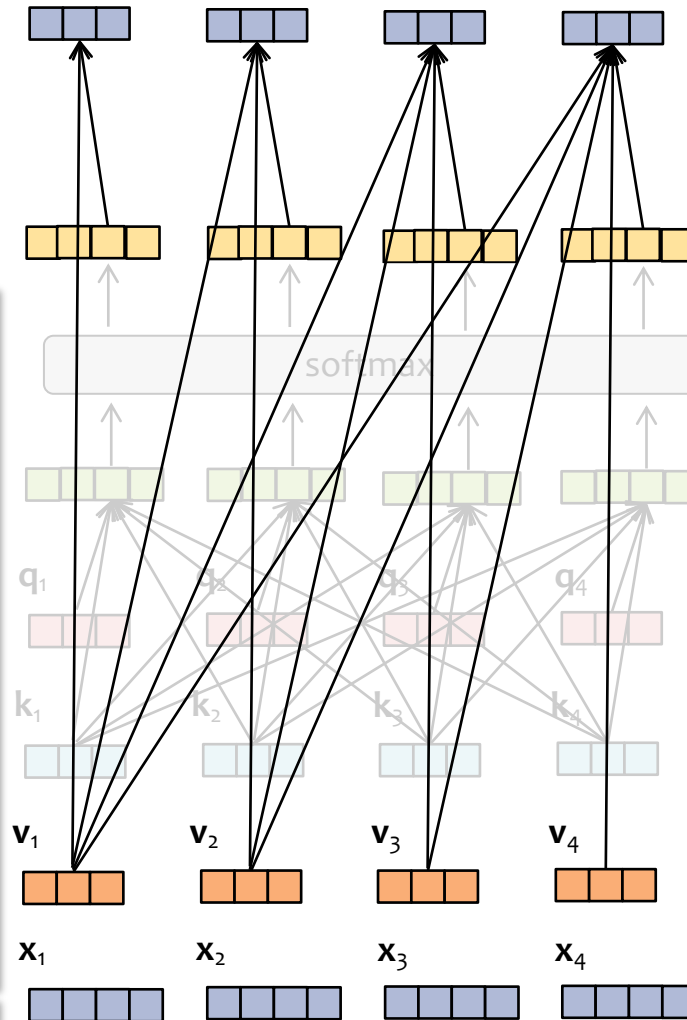
B:

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

C:

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ -\infty & 0 & -\infty & -\infty \\ -\infty & -\infty & 0 & -\infty \\ -\infty & -\infty & -\infty & 0 \end{bmatrix}$$

Answer:



$$\mathbf{X}' = \mathbf{A}\mathbf{V} = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k} + \mathbf{M})\mathbf{V}$$

$$\mathbf{A}_{\text{causal}} = \text{softmax}(\mathbf{S} + \mathbf{M})$$

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^T / \sqrt{d_k}$$

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_q$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}_k$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}_v$$

$$\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_4]^T$$

In practice, the attention weights are computed for all time steps T , then we mask out (by setting to $-\infty$) all the inputs to the softmax that are for the timesteps to the right of the query.

Matrix Version of Single-Headed (Causal) Attention

Insight: if some element in the input to the softmax is $-\infty$, then the corresponding output is 0!

Question: For a causal LM which is the correct matrix?

A:

$$M = \begin{bmatrix} 0 & 0 & 0 & 0 \\ -\infty & 0 & 0 & 0 \\ -\infty & -\infty & 0 & 0 \\ -\infty & -\infty & -\infty & 0 \end{bmatrix}$$

B:

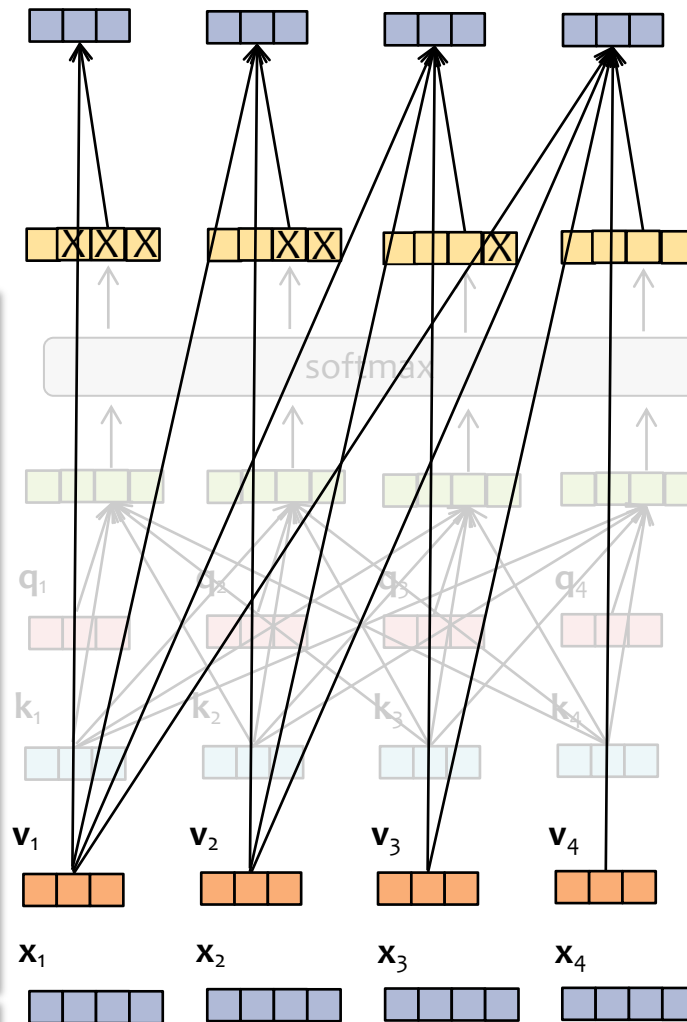
$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

C:

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ -\infty & 0 & -\infty & -\infty \\ -\infty & -\infty & 0 & -\infty \\ -\infty & -\infty & -\infty & 0 \end{bmatrix}$$

Answer:

B



$$\mathbf{X}' = \mathbf{A}\mathbf{V} = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k} + \mathbf{M})\mathbf{V}$$

$$\mathbf{A}_{\text{causal}} = \text{softmax}(\mathbf{S} + \mathbf{M})$$

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^T / \sqrt{d_k}$$

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_q$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}_k$$

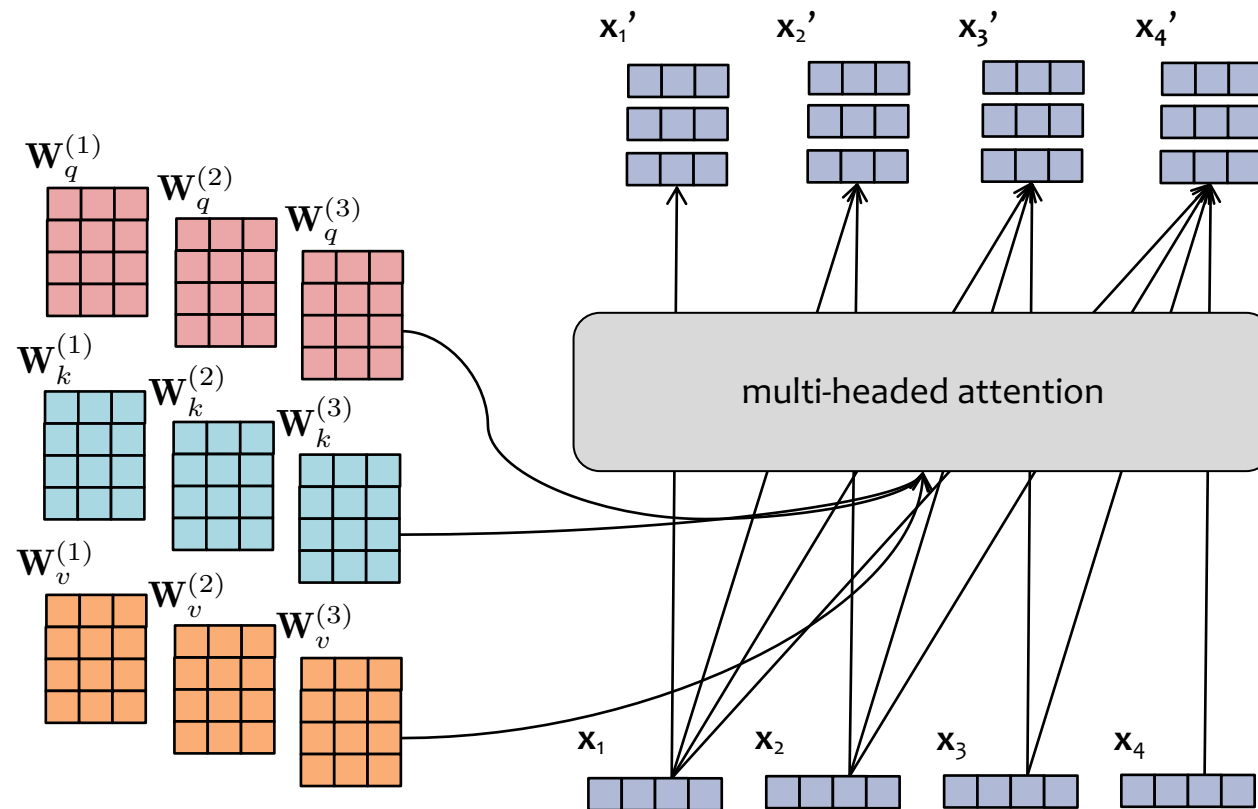
$$\mathbf{V} = \mathbf{X}\mathbf{W}_v$$

$$\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_4]^T$$

In practice, the attention weights are computed for all time steps T , then we mask out (by setting to $-\infty$) all the inputs to the softmax that are for the timesteps to the right of the query.

Matrix Version of Multi-Headed (Causal) Attention

$$\mathbf{X}' = \text{concat}(\mathbf{X}'^{(1)}, \mathbf{X}'^{(2)}, \mathbf{X}'^{(3)})$$



$$\mathbf{X}'^{(i)} = \text{softmax} \left(\frac{\mathbf{Q}^{(i)} (\mathbf{K}^{(i)})^T}{\sqrt{d_k}} + \mathbf{M} \right) \mathbf{V}^{(i)}$$

$$\mathbf{Q}^{(i)} = \mathbf{X} \mathbf{W}_q^{(i)}$$

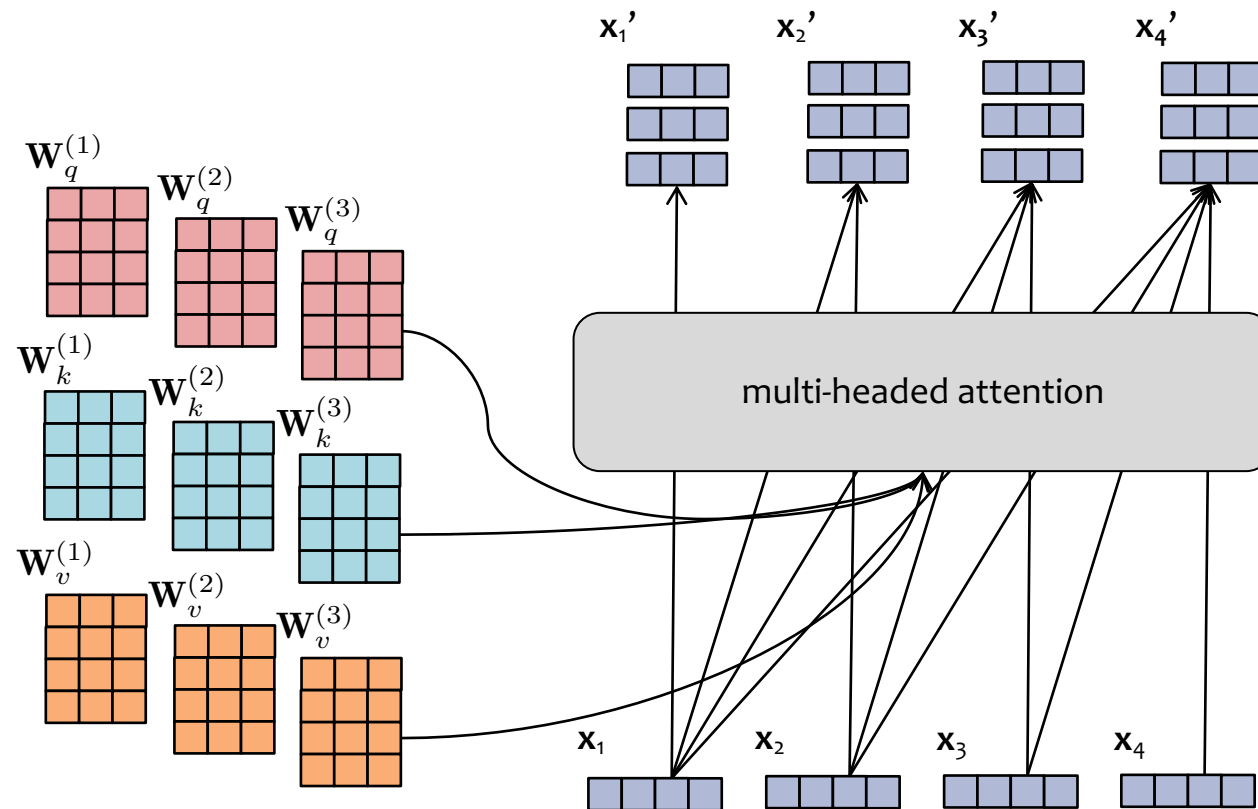
$$\mathbf{K}^{(i)} = \mathbf{X} \mathbf{W}_k^{(i)}$$

$$\mathbf{V}^{(i)} = \mathbf{X} \mathbf{W}_v^{(i)}$$

$$\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_4]^T$$

Matrix Version of Multi-Headed (Causal) Attention

$$\mathbf{X}' = \text{concat}(\mathbf{X}'^{(1)}, \dots, \mathbf{X}'^{(h)})$$



$$\mathbf{X}'^{(i)} = \text{softmax} \left(\frac{\mathbf{Q}^{(i)} (\mathbf{K}^{(i)})^T}{\sqrt{d_k}} + \mathbf{M} \right) \mathbf{V}^{(i)}$$

$$\mathbf{Q}^{(i)} = \mathbf{X} \mathbf{W}_q^{(i)}$$

$$\mathbf{K}^{(i)} = \mathbf{X} \mathbf{W}_k^{(i)}$$

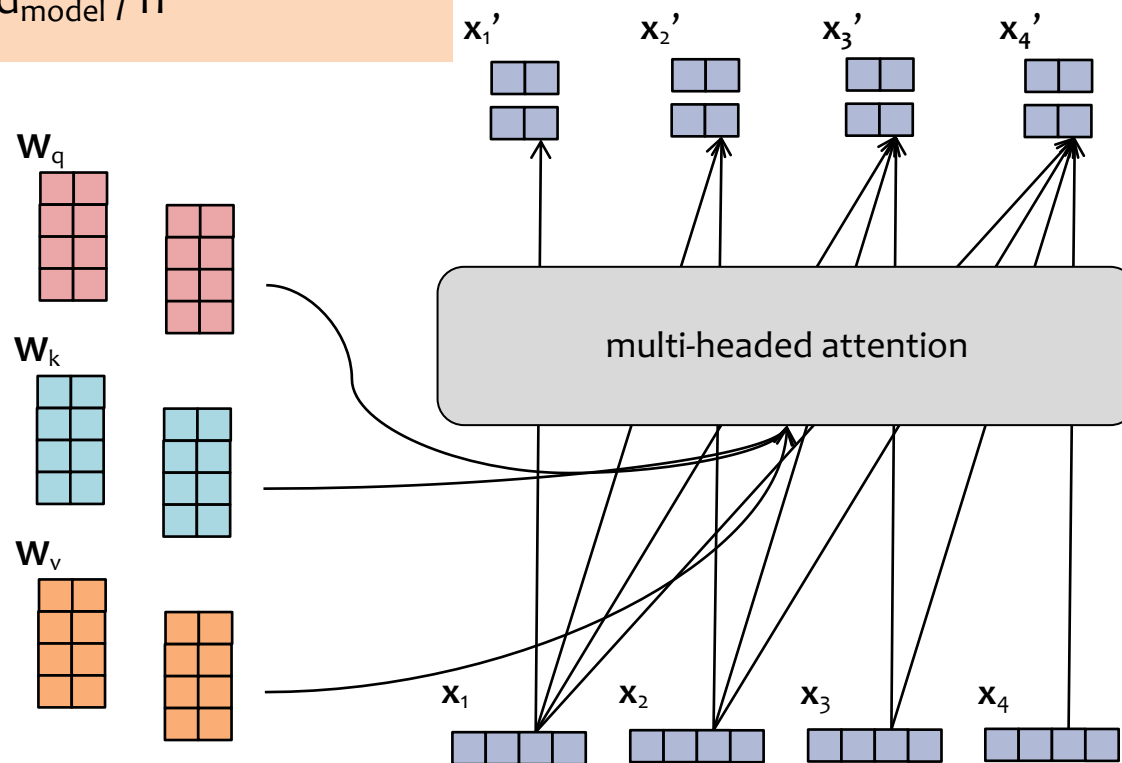
$$\mathbf{V}^{(i)} = \mathbf{X} \mathbf{W}_v^{(i)}$$

$$\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_4]^T$$

Recall:

To ensure the dimension of the **input** embedding $\mathbf{x}_t$ is the same as the **output** embedding $\mathbf{x}_t'$, Transformers usually choose the embedding sizes and number of heads appropriately:

- $d_{\text{model}} = \text{dim. of inputs}$
- $d_k = \text{dim. of each output}$
- $h = \# \text{ of heads}$
- Choose $d_k = d_{\text{model}} / h$



Construction of Multi-Headed (Causal) Attention

$$\mathbf{X}' = \text{concat}(\mathbf{X}'^{(1)}, \dots, \mathbf{X}'^{(h)})$$

$$\mathbf{X}'^{(i)} = \text{softmax} \left(\frac{\mathbf{Q}^{(i)} (\mathbf{K}^{(i)})^T}{\sqrt{d_k}} + \mathbf{M} \right) \mathbf{V}^{(i)}$$

$$\mathbf{Q}^{(i)} = \mathbf{X} \mathbf{W}_q^{(i)}$$

$$\mathbf{K}^{(i)} = \mathbf{X} \mathbf{W}_k^{(i)}$$

$$\mathbf{V}^{(i)} = \mathbf{X} \mathbf{W}_v^{(i)}$$

$$\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_4]^T$$